



Fortify Security Report

Jan 6, 2017

MHussaini

Executive Summary

Issues Overview

On Nov 7, 2016, a source code review was performed over the SpringSecurityEPDS code base. 256 files, 20,260 LOC (Executable) were scanned and reviewed for defects that could lead to potential security vulnerabilities. A total of 105 reviewed findings were uncovered during the analysis.

Issues by Fortify Priority Order

High	70
Critical	35

Recommendations and Conclusions

The Issues Category section provides Fortify recommendations for addressing issues at a generic level. The recommendations for specific fixes can be extrapolated from those generic recommendations by the development group.

Project Summary

Code Base Summary

Code location: C:/AmersFiles/Gitworkspace/SpringSecurityEPDS

Number of Files: 256

Lines of Code: 20260

Build Label: <No Build Label>

Scan Information

Scan time: 06:11

SCA Engine version: 6.30.0086

Machine Name: AIOCOFT55000042

Username running scan: MHussaini

Results Certification

Results Certification Valid

Details:

Results Signature:

SCA Analysis Results has Valid signature

Rules Signature:

There were no custom rules used in this scan

Attack Surface

Attack Surface:

Command Line Arguments:

com.EPDS.US.Test.main

gov.gao.edps.webservice.Test.main

gov.gao.epds.filestorage.FileStorageEncryption.main

gov.gao.epds.filestorage.test.FileEncryptionTest.main

gov.gao.epds.test.MainClass.main

gov.gao.epds.test.ServiceTest.main

gov.gao.epds.test.UploadExecutor.main

gov.gao.epds.test.ValidationTesting.main

gov.gao.epds.testutils.EmailValidatorTest.main

gov.gao.epds.utils.ExportToExcel.main

gov.gao.epds.utils.PropertyFileEncrypter.main

gov.gao.epds.utils.Util_Test.main

gov.gao.epds.utils.templates.HtmlToPDF.main

gov.usda.epds.test.RemoveMapElement.main

gov.usda.epds.test.Sha512Test.main

gov.usda.epds.test.Test.main

File System:

java.io.FileInputStream.FileInputStream

Private Information:

null.null.null
java.security.Provider.getProperty
java.util.Properties.getProperty
javax.crypto.SecretKeyFactory.generateSecret

Java Properties:

java.lang.System.getProperty
java.security.Provider.load
java.util.Properties.load

Stream:

gov.gao.epds.clam.test.ArbitraryInputStream.read
gov.gao.epds.fileupload.utils.ReusableInputStream.read
java.io.BufferedInputStream.read
java.io.FileInputStream.read
java.io.FilterInputStream.read
java.io.InputStream.read
net.lingala.zip4j.io.ZipInputStream.read

System Information:

null.null.null
com.fasterxml.jackson.core.JsonProcessingException.getMessage
java.lang.Throwable.getMessage
java.net.InetAddress.getLocalHost
java.net.URISyntaxException.getMessage
javax.servlet.ServletContext.getRealPath
org.codehaus.jackson.JsonProcessingException.getMessage
org.codehaus.jackson.map.JsonMappingException.getMessage
org.springframework.core.NestedRuntimeException.getMessage

Web:

javax.servlet.ServletRequest.getRemoteAddr

Filter Set Summary

Current Enabled Filter Set:

Quick View

Filter Set Details:

Folder Filters:

If [fortify priority order] contains critical Then set folder to Critical
If [fortify priority order] contains high Then set folder to High
If [fortify priority order] contains medium Then set folder to Medium
If [fortify priority order] contains low Then set folder to Low

Visibility Filters:

If impact is not in range [2.5, 5.0] Then hide issue
If likelihood is not in range (1.0, 5.0] Then hide issue

Audit Guide Summary

Audit guide not enabled

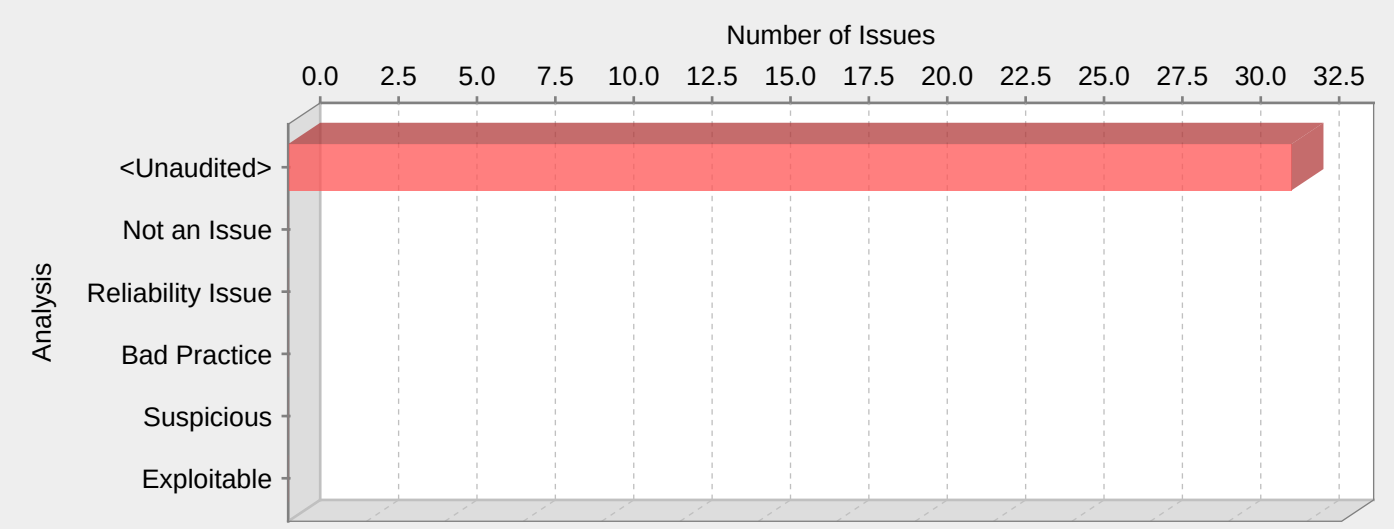
Results Outline

Overall number of results

The scan found 105 issues.

Vulnerability Examples by Category

Category: Password Management: Password in Configuration File (32 Issues)



Explanation:

Storing a plaintext password in a configuration file allows anyone who can read the file access to the password-protected resource. Developers sometimes believe that they cannot defend the application from someone who has access to the configuration, but this attitude makes an attacker's job easier. Good password management guidelines require that a password never be stored in plaintext.

Recommendations:

A password should never be stored in plaintext. Instead, the password should be entered by an administrator when the system starts. If that approach is impractical, a less secure but often adequate solution is to obfuscate the password and scatter the de-obfuscation material around the system so that an attacker has to obtain and correctly combine multiple system resources to decipher the password.

Some third-party products claim the ability to manage passwords in a more secure way. For example, WebSphere Application Server 4.x uses a simple XOR encryption algorithm for obfuscating values, but be skeptical about such facilities. WebSphere and other application servers offer outdated and relatively weak encryption mechanisms that are insufficient for security-sensitive environments. For a secure solution the only viable option is a proprietary one.

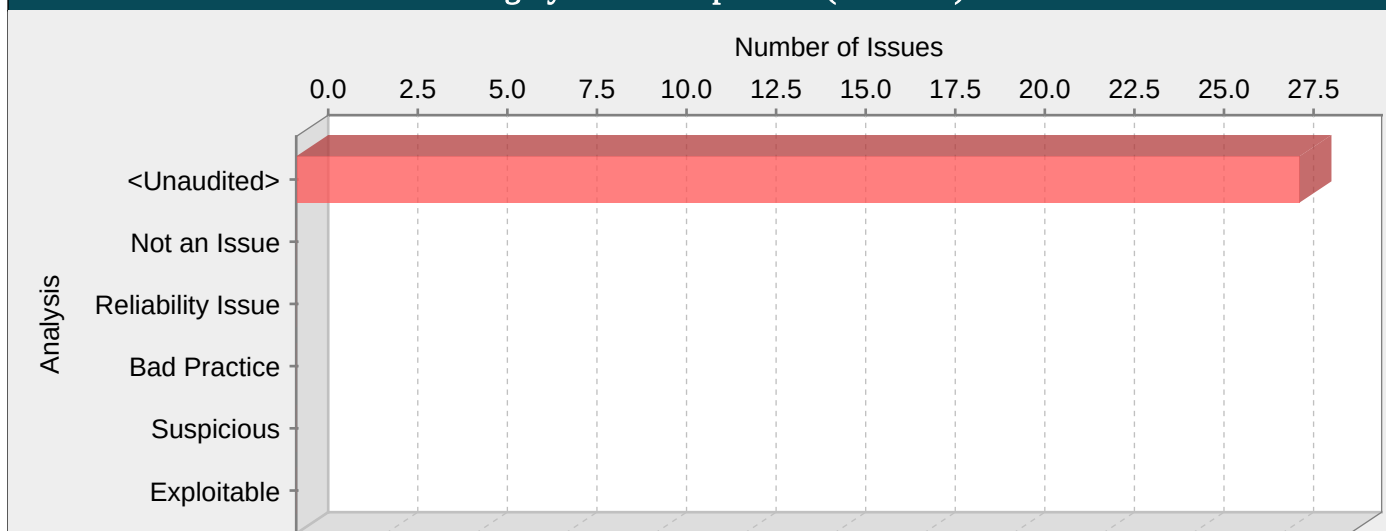
Tips:

- 1. HPE Security Fortify Static Code Analyzer searches configuration files for common names used for password properties. Audit these issues by verifying that the flagged entry is used as a password and that the password entry contains plaintext.
- 2. If the entry in the configuration file is a default password, require that it be changed in addition to requiring that it be obfuscated in the configuration file.

epds.local.properties, line 4 (Password Management: Password in Configuration File)

Fortify Priority:	High	Folder	High
Kingdom:	Environment		
Abstract:	Storing a plaintext password in a configuration file may result in a system compromise.		
Sink:	epds.local.properties:4 forgotPasswordSubmitEmailURI()		
2	#Tue Jun 14 03:43:22 MDT 2016		
3	validateTokenURI=ENC(MKbd7T8BHIhjpezBHPwdX/nVTSuqBwydLxIz6TY1ntZRjLqAgNtjAvaKZaMY0j4lWgSzfwJulhHLvHe+ghUXxt9awmcaicxmLe1bBLnUKHE\=)		
4	forgotPasswordSubmitEmailURI=ENC(Bu4YvzJyLL0ceN3YhwhZq043pvtUZg3qsaJYL4jjvvSgzian7QvDS5zZj/xFHU+iQeXYoDfdNfab9o2eD8BA09LassyHE0Wzvm2IBaZLC8yCBi0cMZe2pQ\=\\=)		
5	checkSecurityAnswer=ENC(8R5QAICKrDbJKuiJ508pYkudBwzvSeNldJIzQV0pb1KvrQEgVdufhKWFzFfx+wniZ2DVau7DSZjGW5nlnCi4NYxd76FXDdYOAcZLY4iVoLx10FdII0dAjob08saqBSKICs46bcKKSfK\=)		
6	validatePasswordURI=ENC(fGXmtNPDsigGzXZDBipobNc/c9Pj0WAawUeCoBX4aAbtnZN9HUJGy1UBa0qd06nbXcIrD10ndkgoAVD/7iL+N63uWInFqPfcwMoxY0chiLu/5J5v24fq1Q\=\\=)		

Category: Path Manipulation (28 Issues)

**Explanation:**

Path manipulation errors occur when the following two conditions are met:

1. An attacker can specify a path used in an operation on the filesystem.
2. By specifying the resource, the attacker gains a capability that would not otherwise be permitted.

For example, the program may give the attacker the ability to overwrite the specified file or run with a configuration controlled by the attacker.

Example 1: The following code uses input from an HTTP request to create a file name. The programmer has not considered the possibility that an attacker could provide a file name such as "../../../tomcat/conf/server.xml", which causes the application to delete one of its own configuration files.

```
String rName = request.getParameter("reportName");
File rFile = new File("/usr/local/apfr/reports/" + rName);
...
rFile.delete();
```

Example 2: The following code uses input from a configuration file to determine which file to open and echo back to the user. If the program runs with privileges and malicious users can change the configuration file, they can use the program to read any file on the system that ends with the extension .txt.

```
fis = new FileInputStream(cfg.getProperty("sub")+".txt");
amt = fis.read(arr);
out.println(arr);
```

Some think that in the mobile world, classic vulnerabilities, such as path manipulation, do not make sense -- why would the user attack themselves? However, keep in mind that the essence of mobile platforms is applications that are downloaded from various sources and run alongside each other on the same device. The likelihood of running a piece of malware next to a banking application is high, which necessitates expanding the attack surface of mobile applications to include inter-process communication.

Example 3: The following code adapts Example 1 to the Android platform.

```
...
String rName = this.getIntent().getExtras().getString("reportName");
File rFile = getBaseContext().getFileStreamPath(rName);
...
rFile.delete();
...
```

Recommendations:

The best way to prevent path manipulation is with a level of indirection: create a list of legitimate resource names that a user is allowed to specify, and only allow the user to select from the list. With this approach the input provided by the user is never used directly to specify the resource name.

In some situations this approach is impractical because the set of legitimate resource names is too large or too hard to keep track of. Programmers often resort to blacklisting in these situations. Blacklisting selectively rejects or escapes potentially dangerous characters before using the input. However, any such list of unsafe characters is likely to be incomplete and will almost certainly become out of date. A better approach is to create a whitelist of characters that are allowed to appear in the resource name and accept input composed exclusively of characters in the approved set.

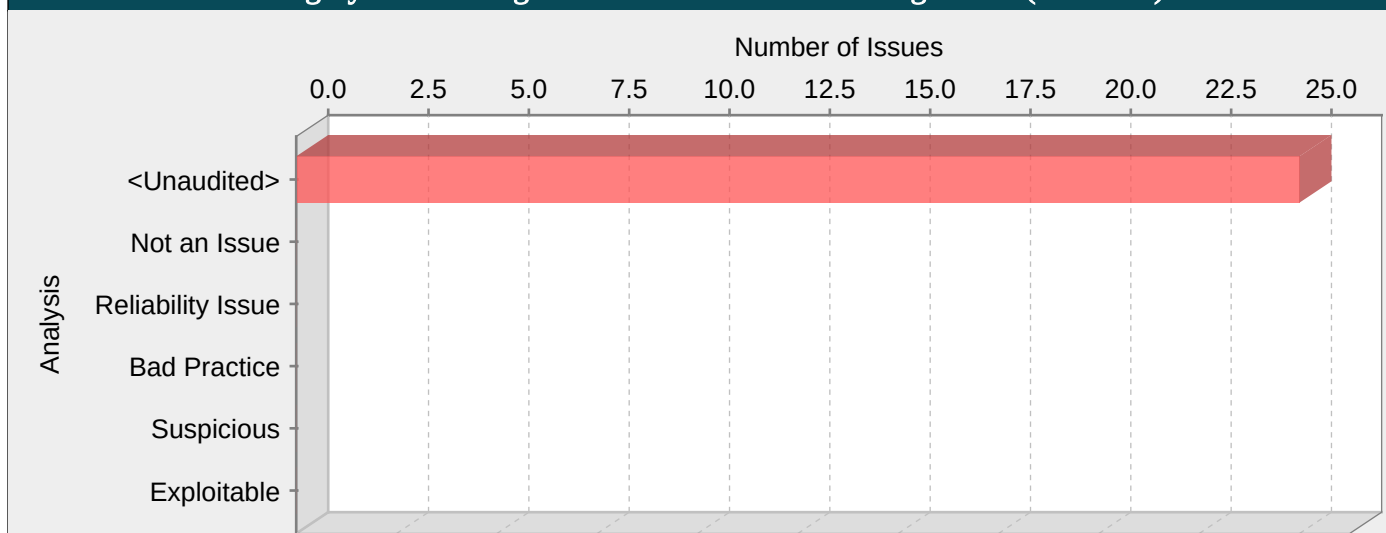
Tips:

- 1. If the program is performing input validation, satisfy yourself that the validation is correct, and use the HPE Security Fortify Custom Rules Editor to create a cleanse rule for the validation routine.
- 2. Since implementing a blacklist that is effective on its own is notoriously difficult, if validation logic relies on blacklisting, one should be skeptical. Consider different types of input encoding and different sets of meta-characters that might have special meaning when interpreted by different operating systems, databases, or other resources. Determine whether or not the blacklist can be updated easily, correctly, and completely if these requirements ever change.
- 3. A number of modern web frameworks provide mechanisms for performing validation of user input. Struts and Spring MVC are among them. To highlight the unvalidated sources of input, the HPE Security Fortify Secure Coding Rulepacks dynamically re-prioritize the issues reported by HPE Security Fortify Static Code Analyzer by lowering their probability of exploit and providing pointers to the supporting evidence whenever the framework validation mechanism is in use. We refer to this feature as Context-Sensitive Ranking. To further assist the HPE Security Fortify user with the auditing process, the HPE Security Fortify Software Security Research group makes available the Data Validation project template that groups the issues into folders based on the validation mechanism applied to their source of input.

EPDSFileUploadController.java, line 287 (Path Manipulation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Input Validation and Representation		
Abstract:	Attackers can control the filesystem path argument to File() at EPDSFileUploadController.java line 287, which allows them to access or modify otherwise protected files.		
Source:	EPDS_FileUtils.java:392 javax.servlet.ServletException.getParameter() 390 if (null != request.getParameter("a_No") 391 && null == a_No) { 392 a_No = request.getParameter("a_No"); 393 }		
Sink:	EPDSFileUploadController.java:287 java.io.File.File() 285 String storage_Path = EPDS_FileUtils.getStoragePath(request, null); 286 287 new File(storage_Path).mkdirs(); 288 String fileStoragePath = new File(storage_Path, fileName) 289 .getAbsolutePath() + ".temp";		

Category: Mass Assignment: Insecure Binder Configuration (25 Issues)

**Explanation:**

To ease development and increase productivity, most modern frameworks allow an object to be automatically instantiated and populated with the HTTP request parameters whose names match an attribute of the class to be bound. Automatic instantiation and population of objects speeds up development, but can lead to serious problems if implemented without caution. Any attribute in the bound classes, or nested classes, will be automatically bound to the HTTP request parameters. Therefore, malicious users will be able to assign a value to any attribute in bound or nested classes, even if they are not exposed to the client through web forms or API contracts.

Example 1: Using Spring MVC with no additional configuration, the following controller method will bind the HTTP request parameters to any attribute in the User or Details classes:

```
@RequestMapping(method = RequestMethod.POST)
public String registerUser(@ModelAttribute("user") User user, BindingResult result, SessionStatus status) {
    if (db.save(user).hasErrors()) {
        return "CustomerForm";
    } else {
        status.setComplete();
        return "CustomerSuccess";
    }
}
```

Where User class is defined as:

```
public class User {
    private String name;
    private String lastname;
    private int age;
    private Details details;
    // Public Getters and Setters
    ...
}
```

and Details class is defined as:

```
public class Details {
    private boolean is_admin;
    private int id;
    private Date login_date;
    // Public Getters and Setters
    ...
}
```

Recommendations:

When using frameworks that provide automatic model binding capabilities, it is a best practice to control which attributes will be bound to the model object so that even if attackers figure out other non-exposed attributes of the model or nested classes, they will not be able to bind arbitrary values from HTTP request parameters.

Depending on the framework used there will be different ways to control the model binding process:

Spring MVC: It is possible to control which HTTP request parameters will be used in the binding process and which ones will be ignored.

Example 2: On Spring applications using annotations, a method can be annotated with `@InitBinder` so that the framework will inject a reference to the Spring Model Binder. The Spring Model Binder can be configured with `setAllowedFields` and `setDisallowedFields` to control which attributes will be bound:

```
final String[] DISALLOWED_FIELDS = new String[]{"details.role", "details.age", "is_admin"};
```

```
@InitBinder
```

```
public void initBinder(WebDataBinder binder) {  
    binder.setDisallowedFields(DISALLOWED_FIELDS);  
}
```

Example 3: Spring applications extending `BaseCommandController` can override the `initBinder(HttpServletRequest request, ServletRequestDataBinder binder)` method in order to get a reference to the Spring model binder:

```
@Override
```

```
protected void initBinder(HttpServletRequest request, ServletRequestDataBinder binder) throws Exception {  
    binder.setDisallowedFields(new String[]{"details.role", "details.age", "is_admin"});  
}
```

Apache Struts: Struts 1 and 2 will only bind HTTP request parameters to those Actions or ActionForms attributes which have an associated public setter accessor. If an attribute should not be bound to the request, its setter should be made private.

Example 4: Configure a private setter so that Struts framework will not automatically bind any HTTP request parameter:

```
private String role;  
private void setRole(String role) {  
    this.role = role;  
}
```

REST frameworks: Most REST frameworks will automatically bind any HTTP request bodies with content type JSON or XML to a model object. Depending on the libraries used for JSON and XML processing, there will be different ways of controlling the binding process. The following are some examples for JAXB (XML) and Jackson (JSON):

Example 5: Models bound from XML documents using Oracle's JAXB library can control the binding process using different annotations such as `@XmlAccessorType`, `@XmlAttribute`, `@XmlElement` and `@XmlTransient`. The binder can be told not to bind any attributes by default, by annotating the models using the `@XmlAccessorType` annotation with the value `XmlAccessType.NONE` and then selecting which fields should be bound using `@XmlAttribute` and `@XmlElement` annotations:

```
@XmlRootElement
```

```
@XmlAccessorType(XmlAccessType.NONE)
```

```
public class User {  
    private String role;  
    private String name;  
    @XmlAttribute  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getRole() {  
        return role;  
    }  
    public void setRole(String role) {  
        this.role = role;  
    }  
}
```

Example 6: Models bound from JSON documents using FastXML's Jackson library can control the binding process using different annotations such as `@JsonIgnore`, `@JsonIgnoreProperties`, `@JsonIgnoreType` and `@JsonInclude`. The binder can be told to ignore certain attributes by annotating them with `@JsonIgnore` annotation:

```
public class User {  
    @JsonIgnore  
    private String role;  
    private String name;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getRole() {  
        return role;  
    }  
    public void setRole(String role) {  
        this.role = role;  
    }  
}
```

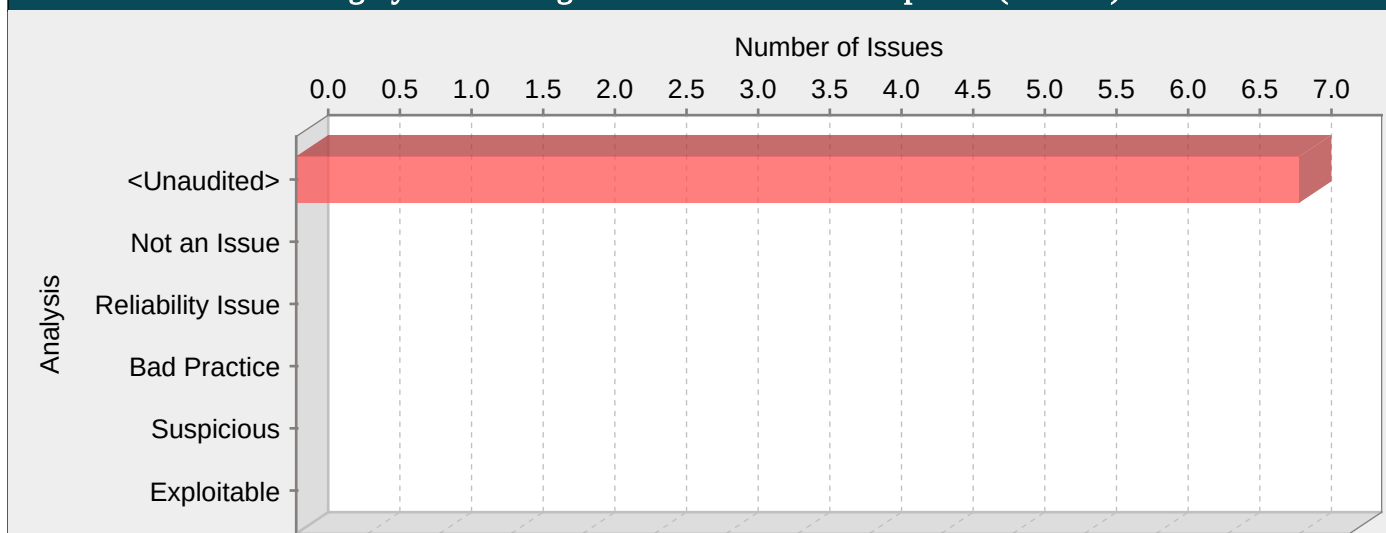
A different approach to protecting against mass assignment vulnerabilities is using a layered architecture where the HTTP request parameters are bound to DTO objects. The DTO objects are only used for that purpose, exposing only those attributes defined in the web forms or API contracts, and then mapping these DTO objects to Domain Objects where the rest of the private attributes can be defined.

Tips:

- 1. This vulnerability category can be classified as a design flaw since accurately finding these issues requires understanding of the application architecture which is beyond the capabilities of static analysis. Therefore, it is possible that if the application is designed to use specific DTO objects for HTTP request binding, there will not be any need to configure the binder to exclude any attributes.

CaseDocketSheetController.java, line 175 (Mass Assignment: Insecure Binder Configuration)			
Fortify Priority:	High	Folder	High
Kingdom:	API Abuse		
Abstract:	The framework binder used for binding the HTTP request parameters to the model class has not been explicitly configured to allow, or disallow certain attributes.		
Sink:	CaseDocketSheetController.java:175 Function: viewCaseDocketFileInfo()		
173	*/		
174	@RequestMapping(value = "/viewcasedocketFileInfo", method = RequestMethod.POST, produces = MediaType.APPLICATION_JSON)		
175	public @ResponseBody ModelMap viewCaseDocketFileInfo(ModelMap map,		
176	/*		
177	* @RequestParam("protestId") String a_No,		

Category: Mass Assignment: Sensitive Field Exposure (7 Issues)

**Explanation:**

Modern frameworks allow developers to automatically bind HTTP request parameters from both request query and body into model objects for ease of development and increased productivity. If the binder is not correctly configured to control which HTTP request parameters are bound to which model attributes, an attacker may be able to abuse the model binding process and set any other attributes that should not be exposed to user control. This binding is possible even if the model attributes do not appear in the web forms or API contracts.

Example 1: The following Spring controller method (registerUser) is accessed from a web form that asks a user to register an account by providing their first name, last name, and age:

```
@RequestMapping(method = RequestMethod.POST)
public String registerUser(@ModelAttribute("user") User user, BindingResult result, SessionStatus status) {
    if (db.save(user).hasErrors()) {
        return "CustomerForm";
    } else {
        status.setComplete();
        return "CustomerSuccess";
    }
}
```

If registration is successful, the user data will be persisted in the database. The User class is defined as:

```
public class User {
    private String name;
    private String lastname;
    private int age;
    private Details details;
    // Public Getters and Setters
    ...
}
```

And the Details class is defined as:

```
public class Details {
    private boolean is_admin;
    private int id;
    private Date login_date;
    // Public Getters and Setters
    ...
}
```

Given the above scenario, an attacker may be able to explore the application and discover that there is a details attribute in the User model. If this is the case, the attacker may then attempt to overwrite the current values assigned to their attributes.

If an attacker can find out these internal attributes, and the framework binder is not correctly configured in order to disallow binding of these attributes, then the attacker would be able to register an administrator account by sending the following request:

```
name=John&lastname=Smith&age=22&details.is_admin=true
```

Recommendations:

In order to avoid mass assignment vulnerabilities it is important to control the HTTP request to model class binding process. There are different alternatives depending on the framework used:

Architectural: Define dedicated DTO classes to be bound to user data and containing only those attributes that should be exposed to end users. Map their attributes with the Domain Objects used along the application. These Domain Objects will contain attributes to receive the validated user data and extra attributes that should never be exposed to user control.

Binder configuration: Certain frameworks such as Spring, allow the developers to configure the model binder to accept or reject HTTP request parameters based on their names. Eg:

Example 2:

```
@Override
```

```
protected void initBinder(HttpServletRequest request, ServletRequestDataBinder binder) throws Exception {
    binder.setDisallowedFields(new String[]{"details.role", "details.age", "is_admin"});
}
```

Model class configuration: Depending on the frameworks used and the serialization/deserialization libraries in use, the model classes to be bound to the HTTP request parameters can be annotated to allow, or disallow, certain attribute bindings. For example, if user data is sent in JSON format and Jackson library is used to deserialize this data, the `@JsonIgnore` annotation can be used to tell Jackson to ignore a given attribute for serialization and deserialization purposes.

Example 3:

```
public class User {
    @JsonIgnore
    private String role;
    private String name;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getRole() {
        return role;
    }
    public void setRole(String role) {
        this.role = role;
    }
}
```

In other frameworks such as Apache Struts 1 and 2, the framework will only allow binding HTTP request parameters to class members with public setters. In case a given attribute should not be exposed, the setter should be declared private.

Example 4:

```
private String role;
private void setRole(String role) {
    this.role = role;
}
```

Tips:

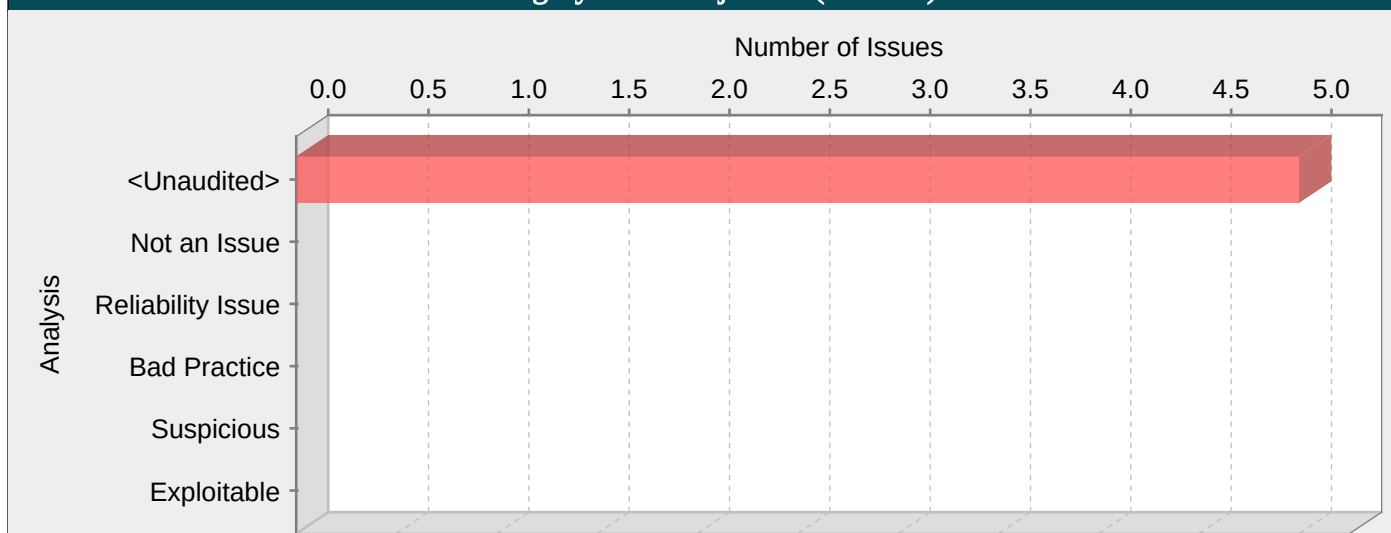
1. This vulnerability category can be classified as a design flaw since accurately finding these issues requires understanding of the application architecture which is beyond the capabilities of static analysis. Therefore, it is possible that if the application is designed to use specific DTO objects for HTTP request binding, there will not be any need to configure the binder to exclude any attributes.

User_info_dto.java, line 422 (Mass Assignment: Sensitive Field Exposure)

Fortify Priority:	High	Folder	High
Kingdom:	Encapsulation		

Abstract:	A sensitive field defined in User_info_dto.java on line 422 is exposed to the model binder.
Source:	User_info_dto.java:421 setRole(0)
419	}
420	
421	public void setRole(String role) {
422	this.role = role;
423	}
Sink:	User_info_dto.java:422 Assignment to this.role()
420	
421	public void setRole(String role) {
422	this.role = role;
423	}

Category: JSON Injection (5 Issues)

**Explanation:**

JSON injection occurs when:

1. Data enters a program from an untrusted source.
2. The data is written to a JSON stream.

Applications typically use JSON to store data or send messages. When used to store data, JSON is often treated like cached data and may potentially contain sensitive information. When used to send messages, JSON is often used in conjunction with a RESTful service and can be used to transmit sensitive information such as authentication credentials.

The semantics of JSON documents and messages can be altered if an application constructs JSON from unvalidated input. In a relatively benign case, an attacker may be able to insert extraneous elements that cause an application to throw an exception while parsing a JSON document or request. In a more serious case, such as that involving JSON injection, an attacker may be able to insert extraneous elements that allow for the predictable manipulation of business critical values within a JSON document or request. In some cases, JSON injection can lead to cross-site scripting or dynamic code evaluation.

Example 1: The following Java code uses Jackson to serialize user account authentication information for non-privileged users (those with a role of "default" as opposed to privileged users with a role of "admin") from user-controlled input variables username and password to the JSON file located at ~/user_info.json:

```
...
JsonFactory jfactory = new JsonFactory();
JsonGenerator jGenerator = jfactory.createJsonGenerator(new File("~/user_info.json"), JsonEncoding.UTF8);
jGenerator.writeStartObject();
jGenerator.writeFieldName("username");
jGenerator.writeRawValue "\"" + username + "\"");
jGenerator.writeFieldName("password");
jGenerator.writeRawValue "\"" + password + "\"");
jGenerator.writeFieldName("role");
jGenerator.writeRawValue "\"default\"");
jGenerator.writeEndObject();
jGenerator.close();
```

Yet, because the JSON serialization is performed using `JsonGenerator.writeRawValue()`, the untrusted data in username and password will not be validated to escape JSON-related special characters. This allows a user to arbitrarily insert JSON keys, possibly changing the structure of the serialized JSON. In this example, if the non-privileged user mallory with password Evil123! were to append `,"role":"admin"` to her username when entering it at the prompt that sets the value of the username variable, the resulting JSON saved to ~/user_info.json would be:

```
{
"username":"mallory",
"role":"admin",
"password":"Evil123!",
```

```
"role":"default"
}
```

If this serialized JSON file were then deserialized to an HashMap object with Jackson's JsonParser as so:

```
JsonParser jParser = jfactory.createJsonParser(new File("~/user_info.json"));
while (jParser.nextToken() != JsonToken.END_OBJECT) {
String fieldname = jParser.getCurrentName();
if ("username".equals(fieldname)) {
jParser.nextToken();
userInfo.put(fieldname, jParser.getText());
}
if ("password".equals(fieldname)) {
jParser.nextToken();
userInfo.put(fieldname, jParser.getText());
}
if ("role".equals(fieldname)) {
jParser.nextToken();
userInfo.put(fieldname, jParser.getText());
}
if (userInfo.size() == 3)
break;
}
jParser.close();
```

The resulting values for the username, password, and role keys in the HashMap object would be mallory, Evil123!, and admin respectively. Without further verification that the deserialized JSON values are valid, the application will incorrectly assign user mallory "admin" privileges.

Recommendations:

When writing user supplied data to JSON some guidelines should be followed:

1. Don't create JSON attributes whose names are derived from user input.
2. Ensure that all serialization to JSON is performed using a safe serialization function that delimits untrusted data within single or double quotes and escapes any special characters.

Example 2: The following Java code implements the same functionality as that in Example 1, but instead uses JsonGenerator.writeString() rather than JsonGenerator.writeRawValue() to serialize the data, therefore ensuring that any untrusted data is properly delimited and escaped:

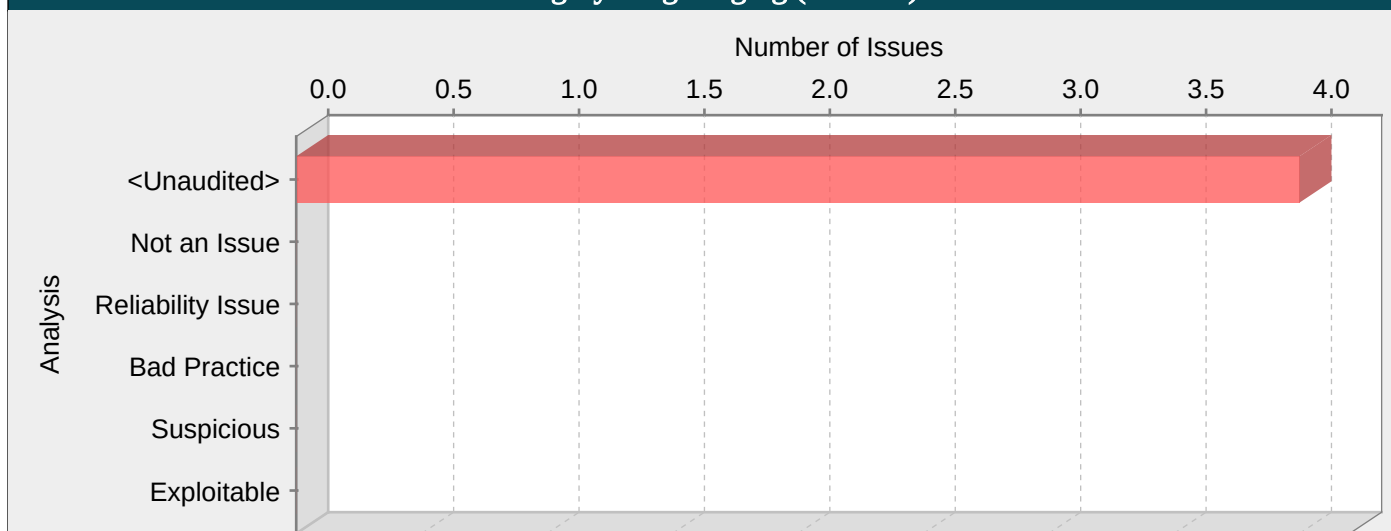
```
...
JsonFactory jfactory = new JsonFactory();
JsonGenerator jGenerator = jfactory.createJsonGenerator(new File("~/user_info.json"), JsonEncoding.UTF8);
jGenerator.writeStartObject();
jGenerator.writeFieldName("username");
jGenerator.writeString(username);
jGenerator.writeFieldName("password");
jGenerator.writeString(password);
jGenerator.writeFieldName("role");
jGenerator.writeString("default");
jGenerator.writeEndObject();
jGenerator.close();
```

AuthUtil.java, line 109 (JSON Injection)

Fortify Priority:	Critical	Folder	Critical
--------------------------	----------	---------------	----------

Kingdom:	Input Validation and Representation
Abstract:	On line 109 of AuthUtil.java, the method getJsonNode() writes unvalidated input into JSON. This call could allow an attacker to inject arbitrary elements or attributes into the JSON entity.
Source:	AuthUtil.java:92 org.springframework.web.client.RestTemplate.postForObject() 90 }else{ 91 jsonResponse = restTemplate 92 .postForObject(PropertyFileEncrypter.decrypt(prop.getProperty(requestURI)), 93 request, String.class); 94 } Sink: AuthUtil.java:109 com.fasterxml.jackson.databind.ObjectMapper.readTree() 107 } 108 109 JsonNode jsonNode = new ObjectMapper().readTree(validResponse); 110 JsonNode message = jsonNode.path(nodePath); 111 return message;

Category: Log Forging (4 Issues)

**Explanation:**

Log forging vulnerabilities occur when:

1. Data enters an application from an untrusted source.
2. The data is written to an application or system log file.

Applications typically use log files to store a history of events or transactions for later review, statistics gathering, or debugging. Depending on the nature of the application, the task of reviewing log files may be performed manually on an as-needed basis or automated with a tool that automatically culls logs for important events or trending information.

Interpretation of the log files may be hindered or misdirected if an attacker can supply data to the application that is subsequently logged verbatim. In the most benign case, an attacker may be able to insert false entries into the log file by providing the application with input that includes appropriate characters. If the log file is processed automatically, the attacker can render the file unusable by corrupting the format of the file or injecting unexpected characters. A more subtle attack might involve skewing the log file statistics. Forged or otherwise, corrupted log files can be used to cover an attacker's tracks or even to implicate another party in the commission of a malicious act [1]. In the worst case, an attacker may inject code or other commands into the log file and take advantage of a vulnerability in the log processing utility [2].

Example 1: The following web application code attempts to read an integer value from a request object. If the value fails to parse as an integer, then the input is logged with an error message indicating what happened.

```
...
String val = request.getParameter("val");
try {
    int value = Integer.parseInt(val);
}
catch (NumberFormatException nfe) {
    log.info("Failed to parse val = " + val);
}
...
```

If a user submits the string "twenty-one" for val, the following entry is logged:

INFO: Failed to parse val=twenty-one

However, if an attacker submits the string "twenty-one%0a%0aINFO:+User+logged+out%3dbadguy", the following entry is logged:

INFO: Failed to parse val=twenty-one

INFO: User logged out=badguy

Clearly, attackers can use this same mechanism to insert arbitrary log entries.

Some think that in the mobile world, classic web application vulnerabilities, such as log forging, do not make sense -- why would the user attack themselves? However, keep in mind that the essence of mobile platforms is applications that are downloaded from various sources and run alongside each other on the same device. The likelihood of running a piece of malware next to a banking application is high, which necessitates expanding the attack surface of mobile applications to include inter-process communication.

Example 2: The following code adapts Example 1 to the Android platform.

```
...
String val = this.getIntent().getExtras().getString("val");
try {
    int value = Integer.parseInt();
}
catch (NumberFormatException nfe) {
    Log.e(TAG, "Failed to parse val = " + val);
}
...
```

Recommendations:

Prevent log forging attacks with indirection: create a set of legitimate log entries that correspond to different events that must be logged and only log entries from this set. To capture dynamic content, such as users logging out of the system, always use server-controlled values rather than user-supplied data. This ensures that the input provided by the user is never used directly in a log entry.

Example 1 can be rewritten to use a pre-defined log entry that corresponds to a `NumberFormatException` as follows:

```
...
public static final String NFE = "Failed to parse val. The input is required to be an integer value."
...
String val = request.getParameter("val");
try {
    int value = Integer.parseInt(val);
}
catch (NumberFormatException nfe) {
    log.info(NFE);
}
..
```

And here is an Android equivalent:

```
...
public static final String NFE = "Failed to parse val. The input is required to be an integer value."
...
String val = this.getIntent().getExtras().getString("val");
try {
    int value = Integer.parseInt();
}
catch (NumberFormatException nfe) {
    Log.e(TAG, NFE);
}
...
```

In some situations this approach is impractical because the set of legitimate log entries is too large or complicated. In these situations, developers often fall back on blacklisting. Blacklisting selectively rejects or escapes potentially dangerous characters before using the input. However, a list of unsafe characters can quickly become incomplete or outdated. A better approach is to create a whitelist of characters that are allowed to appear in log entries and accept input composed exclusively of characters in the approved set. The most critical character in most log forging attacks is the '\n' (newline) character, which should never appear on a log entry whitelist.

Tips:

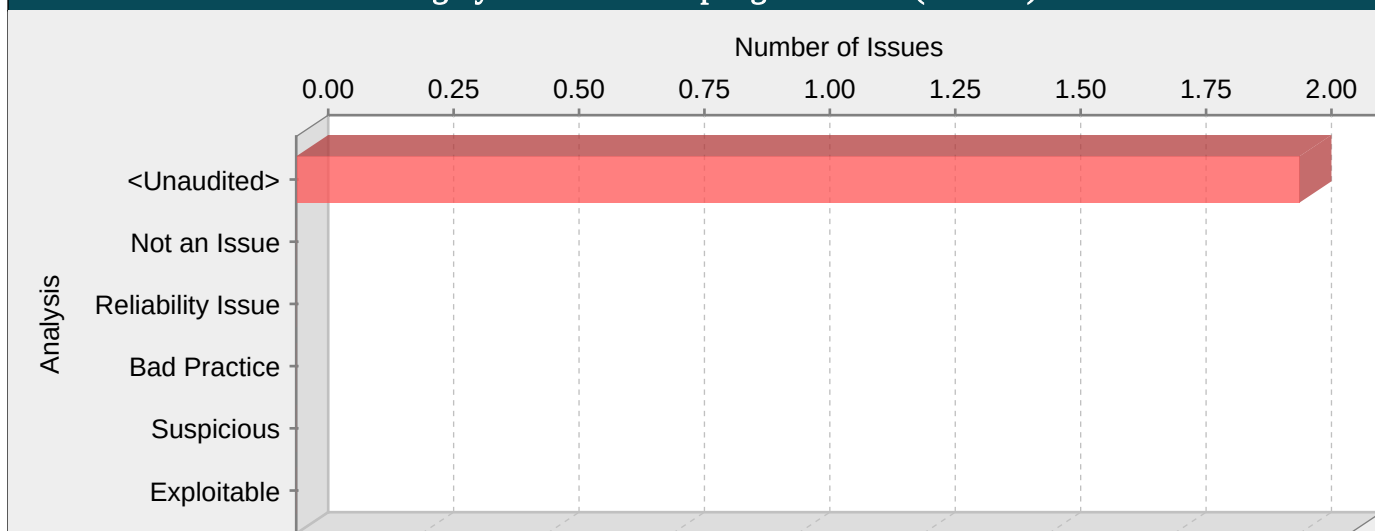
1. Many logging operations are created only for the purpose of debugging a program during development and testing. In our experience, debugging will be enabled, either accidentally or purposefully, in production at some point. Do not excuse log forging vulnerabilities simply because a programmer says "I don't have any plans to turn that on in production".

2. A number of modern web frameworks provide mechanisms for performing validation of user input. Struts and Spring MVC are among them. To highlight the unvalidated sources of input, the HPE Security Fortify Secure Coding Rulepacks dynamically re-prioritize the issues reported by HPE Security Fortify Static Code Analyzer by lowering their probability of exploit and providing pointers to the supporting evidence whenever the framework validation mechanism is in use. We refer to this feature as Context-Sensitive Ranking. To further assist the HPE Security Fortify user with the auditing process, the HPE Security Fortify Software Security Research group makes available the Data Validation project template that groups the issues into folders based on the validation mechanism applied to their source of input.

PreAuthUtil.java, line 40 (Log Forging)

Fortify Priority:	High	Folder	High
Kingdom:	Input Validation and Representation		
Abstract:	The method checIfTokenIsValid() in PreAuthUtil.java writes unvalidated user input to the log on line 40. An attacker could take advantage of this behavior to forge log entries or inject malicious content into the log.		
Source:	Util.java:269 javax.servlet.http.HttpServletRequest.getHeader() 267 } 268 if (ip == null ip.length() == 0 "unknown".equalsIgnoreCase(ip)) { 269 ip = request.getHeader("WL-Proxy-Client-IP"); 270 } 271 if (ip == null ip.length() == 0 "unknown".equalsIgnoreCase(ip)) {		
Sink:	PreAuthUtil.java:40 org.slf4j.Logger.info() 38 boolean isValid = false; 39 String response = ""; 40 logger.info("Validate Token response clientIp ={}",remoteIp); 41 42 try {		

Category: Cross-Site Scripting: Reflected (2 Issues)

**Explanation:**

Cross-site scripting (XSS) vulnerabilities occur when:

1. Data enters a web application through an untrusted source. In the case of Reflected XSS, the untrusted source is typically a web request, while in the case of Persisted (also known as Stored) XSS it is typically a database or other back-end datastore.
2. The data is included in dynamic content that is sent to a web user without being validated.

The malicious content sent to the web browser often takes the form of a segment of JavaScript, but may also include HTML, Flash or any other type of code that the browser may execute. The variety of attacks based on XSS is almost limitless, but they commonly include transmitting private data like cookies or other session information to the attacker, redirecting the victim to web content controlled by the attacker, or performing other malicious operations on the user's machine under the guise of the vulnerable site.

Example 1: The following JSP code segment reads an employee ID, `eid`, from an HTTP request and displays it to the user.

```
<% String eid = request.getParameter("eid"); %>
```

```
...
```

```
Employee ID: <%= eid %>
```

The code in this example operates correctly if `eid` contains only standard alphanumeric text. If `eid` has a value that includes meta-characters or source code, then the code will be executed by the web browser as it displays the HTTP response.

Initially this might not appear to be much of a vulnerability. After all, why would someone enter a URL that causes malicious code to run on their own computer? The real danger is that an attacker will create the malicious URL, then use e-mail or social engineering tricks to lure victims into visiting a link to the URL. When victims click the link, they unwittingly reflect the malicious content through the vulnerable web application back to their own computers. This mechanism of exploiting vulnerable web applications is known as Reflected XSS.

Example 2: The following JSP code segment queries a database for an employee with a given ID and prints the corresponding employee's name.

```
<%...
```

```
Statement stmt = conn.createStatement();
```

```
ResultSet rs = stmt.executeQuery("select * from emp where id="+eid);
```

```
if (rs != null) {
```

```
rs.next();
```

```
String name = rs.getString("name");
```

```
}
```

```
%>
```

```
Employee Name: <%= name %>
```

As in Example 1, this code functions correctly when the values of name are well-behaved, but it does nothing to prevent exploits if they are not. Again, this code can appear less dangerous because the value of name is read from a database, whose contents are apparently managed by the application. However, if the value of name originates from user-supplied data, then the database can be a conduit for malicious content. Without proper input validation on all data stored in the database, an attacker can execute malicious commands in the user's web browser. This type of exploit, known as Persistent (or Stored) XSS, is particularly insidious because the indirection caused by the data store makes it more difficult to identify the threat and increases the possibility that the attack will affect multiple users. XSS got its start in this form with web sites that offered a "guestbook" to visitors. Attackers would include JavaScript in their guestbook entries, and all subsequent visitors to the guestbook page would execute the malicious code.

Some think that in the mobile world, classic web application vulnerabilities, such as cross-site scripting, do not make sense -- why would the user attack themselves? However, keep in mind that the essence of mobile platforms is applications that are downloaded from various sources and run alongside each other on the same device. The likelihood of running a piece of malware next to a banking application is high, which necessitates expanding the attack surface of mobile applications to include inter-process communication.

Example 3: The following code enables JavaScript in Android's WebView (by default, JavaScript is disabled) and loads a page based on the value received from an Android intent.

```
...
WebView webview = (WebView) findViewById(R.id.webview);
webview.getSettings().setJavaScriptEnabled(true);
String url = this getIntent().getExtras().getString("url");
webview.loadUrl(url);
...
```

If the value of url starts with javascript:, JavaScript code that follows will execute within the context of the web page inside WebView.

As the examples demonstrate, XSS vulnerabilities are caused by code that includes unvalidated data in an HTTP response. There are three vectors by which an XSS attack can reach a victim:

- As in Example 1, data is read directly from the HTTP request and reflected back in the HTTP response. Reflected XSS exploits occur when an attacker causes a user to supply dangerous content to a vulnerable web application, which is then reflected back to the user and executed by the web browser. The most common mechanism for delivering malicious content is to include it as a parameter in a URL that is posted publicly or e-mailed directly to victims. URLs constructed in this manner constitute the core of many phishing schemes, whereby an attacker convinces victims to visit a URL that refers to a vulnerable site. After the site reflects the attacker's content back to the user, the content is executed and proceeds to transfer private information, such as cookies that may include session information, from the user's machine to the attacker or perform other nefarious activities.

- As in Example 2, the application stores dangerous data in a database or other trusted data store. The dangerous data is subsequently read back into the application and included in dynamic content. Persistent XSS exploits occur when an attacker injects dangerous content into a data store that is later read and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

- As in Example 3, a source outside the application stores dangerous data in a database or other data store, and the dangerous data is subsequently read back into the application as trusted data and included in dynamic content.

A number of modern web frameworks provide mechanisms for performing validation of user input. Struts and Spring MVC are among them. To highlight the unvalidated sources of input, the rulepacks dynamically re-prioritize the issues reported by HPE Security Fortify Static Code Analyzer by lowering their probability of exploit and providing pointers to the supporting evidence whenever the framework validation mechanism is in use. We refer to this feature as Context-Sensitive Ranking. To further assist the HPE Security Fortify user with the auditing process, the HPE Security Fortify Software Security Research group makes available the Data Validation project template that groups the issues into folders based on the validation mechanism applied to their source of input.

Recommendations:

The solution to XSS is to ensure that validation occurs in the correct places and checks for the correct properties.

Since XSS vulnerabilities occur when an application includes malicious data in its output, one logical approach is to validate data immediately before it leaves the application. However, because web applications often have complex and intricate code for generating dynamic content, this method is prone to errors of omission (missing validation). An effective way to mitigate this risk is to also perform input validation for XSS.

Web applications must validate their input to prevent other vulnerabilities, such as SQL injection, so augmenting an application's existing input validation mechanism to include checks for XSS is generally relatively easy. Despite its value, input validation for XSS does not take the place of rigorous output validation. An application may accept input through a shared data store or other trusted source, and that data store may accept input from a source that does not perform adequate input validation. Therefore, the application cannot implicitly rely on the safety of this or any other data. This means the best way to prevent XSS vulnerabilities is to validate everything that enters the application and leaves the application destined for the user.

The most secure approach to validation for XSS is to create a whitelist of safe characters that are allowed to appear in HTTP content and accept input composed exclusively of characters in the approved set. For example, a valid username might only include alpha-numeric characters or a phone number might only include digits 0-9. However, this solution is often infeasible in web applications because many characters that have special meaning to the browser should still be considered valid input once they are encoded, such as a web design bulletin board that must accept HTML fragments from its users.

A more flexible, but less secure approach is known as blacklisting, which selectively rejects or escapes potentially dangerous characters before using the input. In order to form such a list, you first need to understand the set of characters that hold special meaning for web browsers. Although the HTML standard defines what characters have special meaning, many web browsers try to correct common mistakes in HTML and may treat other characters as special in certain contexts, which is why we do not encourage the use of blacklists as a means to prevent XSS. The CERT(R) Coordination Center at the Software Engineering Institute at Carnegie Mellon University provides the following details about special characters in various contexts [1]:

In the content of a block-level element (in the middle of a paragraph of text):

- "<" is special because it introduces a tag.
- "&" is special because it introduces a character entity.
- ">" is special because some browsers treat it as special, on the assumption that the author of the page intended to include an opening "<", but omitted it in error.

The following principles apply to attribute values:

- In attribute values enclosed with double quotes, the double quotes are special because they mark the end of the attribute value.
- In attribute values enclosed with single quote, the single quotes are special because they mark the end of the attribute value.
- In attribute values without any quotes, white-space characters, such as space and tab, are special.
- "&" is special when used with certain attributes, because it introduces a character entity.

In URLs, for example, a search engine might provide a link within the results page that the user can click to re-run the search. This can be implemented by encoding the search query inside the URL, which introduces additional special characters:

- Space, tab, and new line are special because they mark the end of the URL.
- "&" is special because it either introduces a character entity or separates CGI parameters.
- Non-ASCII characters (that is, everything above 128 in the ISO-8859-1 encoding) are not allowed in URLs, so they are considered to be special in this context.
- The "%" symbol must be filtered from input anywhere parameters encoded with HTTP escape sequences are decoded by server-side code. For example, "%" must be filtered if input such as "%68%65%6C%6C%6F" becomes "hello" when it appears on the web page in question.

Within the body of a <SCRIPT> </SCRIPT>:

- Semicolons, parentheses, curly braces, and new line characters should be filtered out in situations where text could be inserted directly into a pre-existing script tag.

Server-side scripts:

- Server-side scripts that convert any exclamation characters (!) in input to double-quote characters (") on output might require additional filtering.

Other possibilities:

- If an attacker submits a request in UTF-7, the special character '<' appears as '+ADw-' and may bypass filtering. If the output is included in a page that does not explicitly specify an encoding format, then some browsers try to intelligently identify the encoding based on the content (in this case, UTF-7).

Once you identify the correct points in an application to perform validation for XSS attacks and what special characters the validation should consider, the next challenge is to identify how your validation handles special characters. If special characters are not considered valid input to the application, then you can reject any input that contains special characters as invalid. A second option in this situation is to remove special characters with filtering. However, filtering has the side effect of changing any visual representation of the filtered content and may be unacceptable in circumstances where the integrity of the input must be preserved for display.

If input containing special characters must be accepted and displayed accurately, validation must encode any special characters to remove their significance. A complete list of ISO 8859-1 encoded values for special characters is provided as part of the official HTML specification [2].

Many application servers attempt to limit an application's exposure to cross-site scripting vulnerabilities by providing implementations for the functions responsible for setting certain specific HTTP response content that perform validation for the characters essential to a cross-site scripting attack. Do not rely on the server running your application to make it secure. When an application is developed there are no guarantees about what application servers it will run on during its lifetime. As standards and known exploits evolve, there are no guarantees that application servers will also stay in sync.

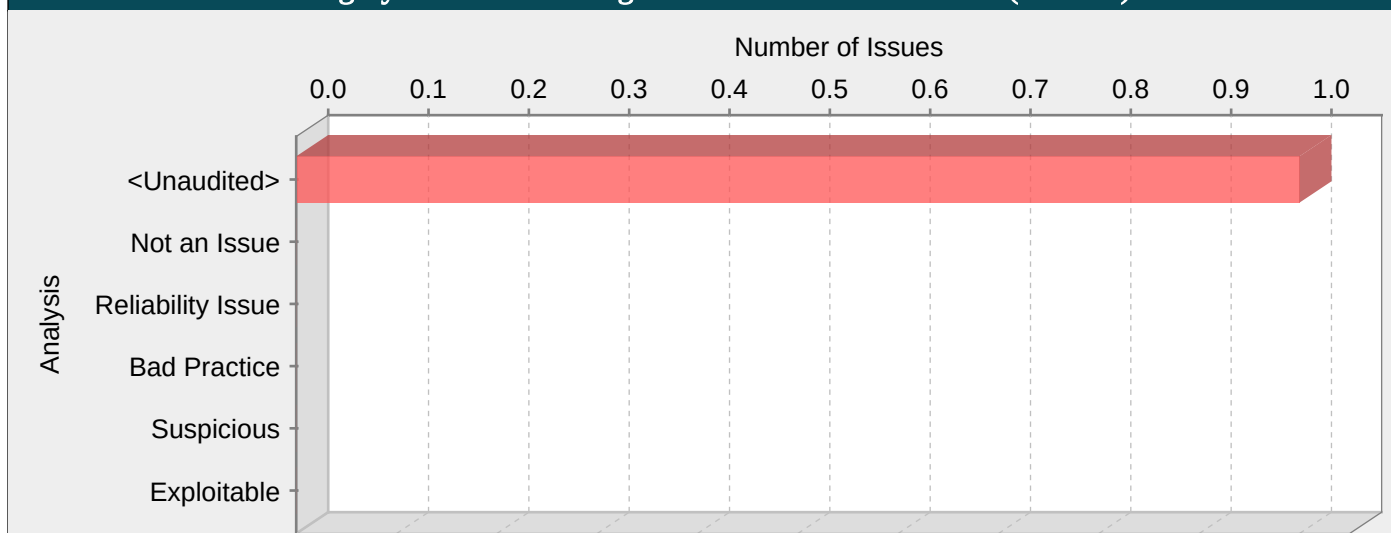
Tips:

- 1. The HPE Security Fortify Secure Coding Rulepacks warn about SQL Injection and Access Control: Database issues when untrusted data is written to a database and also treat the database as a source of untrusted data, which can lead to XSS vulnerabilities. If the database is a trusted resource in your environment, use custom filters to filter out dataflow issues that include the DATABASE taint flag or originate from database sources. Nonetheless, it is often still a good idea to validate everything read from the database.
- 2. Even though URL encoding untrusted data protects against many XSS attacks, some browsers (specifically, Internet Explorer 6 and 7 and possibly others) automatically decode content at certain locations within the Document Object Model (DOM) prior to passing it to the JavaScript interpreter. To reflect this danger, the rulepacks no longer treat URL encoding routines as sufficient to protect against cross-site scripting. Data values that are URL encoded and subsequently output will cause Fortify to report Cross-Site Scripting: Poor Validation vulnerabilities.
- 3. Fortify RTA adds protection against this category.

EPDSFileUploadController.java, line 114 (Cross-Site Scripting: Reflected)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Input Validation and Representation		
Abstract:	The method doPost() in EPDSFileUploadController.java sends unvalidated data to a web browser on line 114, which can result in the browser executing malicious code.		
Source:	EPDSFileUploadController.java:73 doPost(2)		
71	@RequestMapping(value = "/upload-epds-documents", method = RequestMethod.POST)		
72	protected void doPost(HttpServletRequest request,		
73	HttpServletResponse response, FileUploadDTO dto) throws ServletException, IOException		
	{		
74			
75	Map<String,Object> errorMap = new HashMap<String,Object>();		
Sink:	EPDSFileUploadController.java:114 java.io.PrintWriter.write()		
112			
113	PrintWriter out = response.getWriter();		
114	out.write(jsonResponse);		
115	out.flush();		
116			

Category: Password Management: Hardcoded Password (1 Issues)

**Explanation:**

It is never a good idea to hardcode a password. Not only does hardcoding a password allow all of the project's developers to view the password, it also makes fixing the problem extremely difficult. Once the code is in production, the password cannot be changed without patching the software. If the account protected by the password is compromised, the owners of the system will be forced to choose between security and availability.

Example 1: The following code uses a hardcoded password to connect to a database:

```
...
DriverManager.getConnection(url, "scott", "tiger");
...
```

This code will run successfully, but anyone who has access to it will have access to the password. Once the program has shipped, there is no going back from the database user "scott" with a password of "tiger" unless the program is patched. A devious employee with access to this information can use it to break into the system. Even worse, if attackers have access to the bytecode for the application they can use the `javap -c` command to access the disassembled code, which will contain the values of the passwords used. The result of this operation might look something like the following for the example above:

```
javap -c ConnMngr.class
22: ldc  #36; //String jdbc:mysql://ixne.com/rxsql
24: ldc  #38; //String scott
26: ldc  #17; //String tiger
```

In the mobile world, password management is even trickier, considering a much higher chance of device loss.

Example 2: The code below uses hardcoded username and password to setup authentication for viewing protected pages with Android's WebView.

```
...
webview.setWebViewClient(new WebViewClient() {
public void onReceivedHttpAuthRequest(WebView view,
HttpAuthHandler handler, String host, String realm) {
handler.proceed("guest", "allow");
}
});
...
```

Similar to Example 1, this code will run successfully, but anyone who has access to it will have access to the password.

Recommendations:

Passwords should never be hardcoded and should generally be obfuscated and managed in an external source. Storing passwords in plaintext anywhere on the system allows anyone with sufficient permissions to read and potentially misuse the password. At the very least, passwords should be hashed before being stored.

Some third-party products claim the ability to manage passwords in a more secure way. For example, WebSphere Application Server 4.x uses a simple XOR encryption algorithm for obfuscating values, but be skeptical about such facilities. WebSphere and other application servers offer outdated and relatively weak encryption mechanisms that are insufficient for security-sensitive environments. For a secure generic solution, the best option today appears to be a proprietary mechanism that you create.

For Android, as well as any other platform that uses SQLite database, a good option is SQLCipher -- an extension to SQLite database that provides transparent 256-bit AES encryption of database files. Thus, credentials can be stored in an encrypted database.

Example 3: The code below demonstrates how to integrate SQLCipher into an Android application after downloading the necessary binaries, and store credentials into the database file.

```
import net.sqlcipher.database.SQLiteDatabase;
...
SQLiteDatabase.loadLibs(this);
File dbFile = getDatabasePath("credentials.db");
dbFile.mkdirs();
dbFile.delete();
SQLiteDatabase db = SQLiteDatabase.openOrCreateDatabase(dbFile, "credentials", null);
db.execSQL("create table credentials(u, p)");
db.execSQL("insert into credentials(u, p) values(?, ?)", new Object[]{username, password});
...
```

Note that references to android.database.sqlite.SQLiteDatabase are substituted with those of net.sqlcipher.database.SQLiteDatabase.

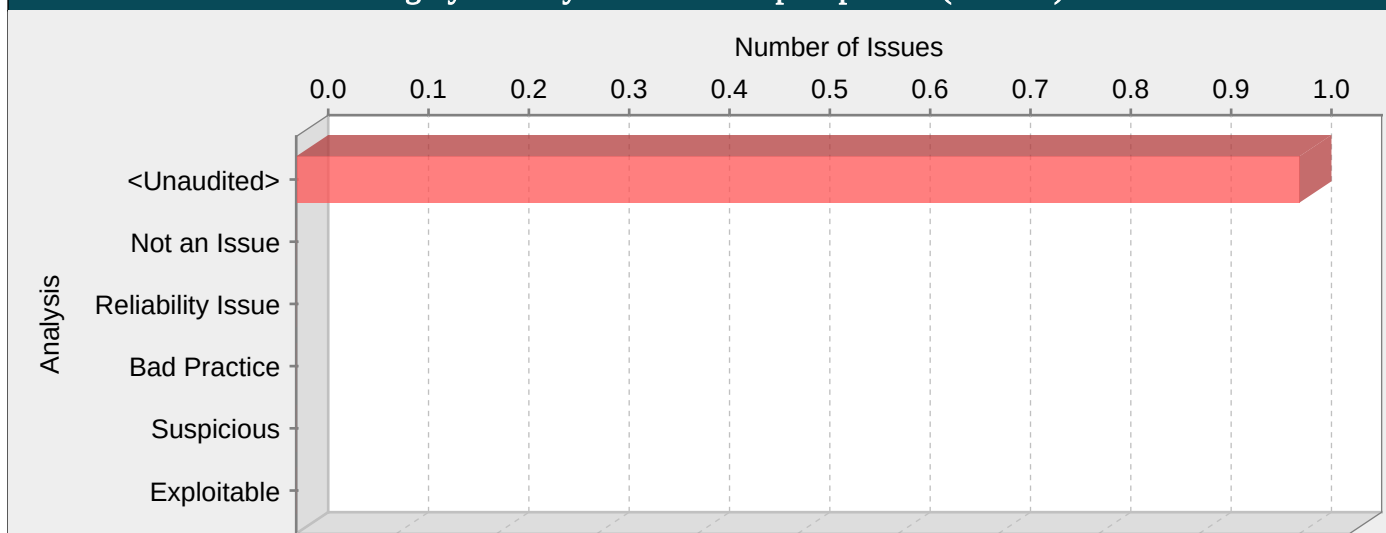
To enable encryption on the WebView store, WebKit has to be re-compiled with the sqlcipher.so library.

Tips:

- 1. The Fortify Java Annotations FortifyPassword and FortifyNotPassword can be used to indicate which fields and variables represent passwords.
- 2. When identifying null, empty, or hardcoded passwords, default rules only consider fields and variables that contain the word password. However, the HPE Security Fortify Custom Rules Editor provides the Password Management wizard that makes it easy to create rules for detecting password management issues on custom-named fields and variables.

Test.java, line 41 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords may compromise system security in a way that cannot be easily remedied.		
Sink:	Test.java:41 FieldAccess: PASSWORD_HASH_ALGORITHM()		
39	public static final String CIPHER_ALGORITHM = "AES";		
40	public static final String KEY_ALGORITHM = "AES";		
41	public static final String PASSWORD_HASH_ALGORITHM = "SHA-256";		
42	public static final String baseUrl =		
	"http://localhost:8080/SpringSecurityEPDS/rest/gc-track-services";		
43	/*public static final String baseUrl		
	="https://epdtest.edc.usda.gov/SpringSecurityEPDS1/rest/gc-track-services/";*/		

Category: Privacy Violation: Heap Inspection (1 Issues)

**Explanation:**

Sensitive data (such as passwords, social security numbers, credit card numbers etc) stored in memory can be leaked if memory is not cleared after use. Often, Strings are used store sensitive data, however, since String objects are immutable, removing the value of a String from memory can only be done by the JVM garbage collector. The garbage collector is not required to run unless the JVM is low on memory, so there is no guarantee as to when garbage collection will take place. In the event of an application crash, a memory dump of the application might reveal sensitive data.

Example 1: The following code converts a password from a character array to a String.

```
private JPasswordField pf;
...
final char[] password = pf.getPassword();
...
String passwordAsString = new String(password);
```

This category was derived from the Cigital Java Rulepack. <http://www.cigital.com/>

Recommendations:

Always be sure to clear sensitive data when it is no longer needed. Instead of storing sensitive data in immutable objects like Strings, use byte arrays or character arrays that can be programmatically cleared.

Example 2: The following code clears memory after a password is used.

```
private JPasswordField pf;
...
final char[] password = pf.getPassword();
// use the password
...
// erase when finished
Arrays.fill(password, '');
```

Tips:

1. A number of modern web frameworks provide mechanisms for performing validation of user input. Struts and Spring MVC are among them. To highlight the unvalidated sources of input, the HPE Security Fortify Secure Coding Rulepacks dynamically re-prioritize the issues reported by HPE Security Fortify Static Code Analyzer by lowering their probability of exploit and providing pointers to the supporting evidence whenever the framework validation mechanism is in use. We refer to this feature as Context-Sensitive Ranking. To further assist the HPE Security Fortify user with the auditing process, the HPE Security Fortify Software Security Research group makes available the Data Validation project template that groups the issues into folders based on the validation mechanism applied to their source of input.

Test.java, line 391 (Privacy Violation: Heap Inspection)

Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	The method buildKey() in Test.java stores sensitive data in a String object, making it impossible to reliably purge the data from memory.		
Source:	Test.java:391 Read password()		
389			

```
390         MessageDigest digester = MessageDigest.getInstance(PASSWORD_HASH_ALGORITHM, "BC");
391         digester.update(String.valueOf(password).getBytes("UTF-8"));
392         byte[] key = digester.digest();
393         SecretKeySpec spec = new SecretKeySpec(key, KEY_ALGORITHM);
Sink:      Test.java:391 java.lang.String.valueOf()
389
390         MessageDigest digester = MessageDigest.getInstance(PASSWORD_HASH_ALGORITHM, "BC");
391         digester.update(String.valueOf(password).getBytes("UTF-8"));
392         byte[] key = digester.digest();
393         SecretKeySpec spec = new SecretKeySpec(key, KEY_ALGORITHM);
```

Detailed Project Summary

Files Scanned

Code base location: C:/AmersFiles/Gitworkspace/SpringSecurityEPDS

Files Scanned:

.settings/org.eclipse.wst.common.project.facet.core.prefs.xml xml 471 bytes Aug 9, 2016 1:44:47 PM
.settings/org.eclipse.wst.common.project.facet.core.xml xml 536 bytes Aug 23, 2016 3:04:46 PM
functional-tests/pom.xml xml 5.4 KB Feb 10, 2016 3:37:54 PM
functional-tests/src/test/java/org/jboss/as/quickstarts/kitchensink/test/Deployments.java java 3 Lines 1.6 KB Feb 10, 2016 3:37:54 PM
functional-tests/src/test/java/org/jboss/as/quickstarts/kitchensink/test/KitchensinkSpringBasicTest.java java 48 Lines 7.1 KB Feb 10, 2016 3:37:54 PM
functional-tests/src/test/java/org/jboss/as/quickstarts/kitchensink/test/Member.java java 21 Lines 2.3 KB Feb 10, 2016 3:37:54 PM
functional-tests/src/test/java/org/jboss/as/quickstarts/kitchensink/test/RESTTest.java java 15 Lines 3 KB Feb 10, 2016 3:37:54 PM
functional-tests/src/test/java/org/jboss/as/quickstarts/kitchensink/test/page/MemberPageFragment.java java 2 Lines 1.3 KB Feb 10, 2016 3:37:54 PM
functional-tests/src/test/java/org/jboss/as/quickstarts/kitchensink/test/page/MembersTablePageFragment.java java 3 Lines 1.3 KB Feb 10, 2016 3:37:54 PM
functional-tests/src/test/java/org/jboss/as/quickstarts/kitchensink/test/page/RegistrationFormPageFragment.java java 12 Lines 2.9 KB Feb 10, 2016 3:37:54 PM
functional-tests/src/test/resources/arquillian.xml xml 2.1 KB Feb 10, 2016 3:37:54 PM
pom.xml xml 13.9 KB Oct 3, 2016 3:11:03 PM
src/main/java/gov/gao/epds/clamscan/ClamScan.java java 127 Lines 8.9 KB Jun 13, 2016 8:39:51 AM
src/main/java/gov/gao/epds/clamscan/ScanResult.java java 27 Lines 2 KB Oct 27, 2016 3:55:38 PM
src/main/java/gov/gao/epds/config/HibernateConfiguration.java java 2.6 KB May 19, 2016 9:17:55 AM
src/main/java/gov/gao/epds/config/JaversSpringJpaApplicationConfig.java java 5.2 KB May 20, 2016 10:08:01 AM
src/main/java/gov/gao/epds/config/PropertyFileConfig.java java 4 Lines 773 bytes Apr 15, 2016 11:53:01 AM
src/main/java/gov/gao/epds/controller/CaseDocketSheetController.java java 219 Lines 31.2 KB Nov 7, 2016 7:33:51 AM
src/main/java/gov/gao/epds/controller/DashboardController.java java 184 Lines 21.2 KB Oct 31, 2016 1:40:26 PM
src/main/java/gov/gao/epds/controller/EPDSFileUploadController.java java 124 Lines 12 KB Sep 27, 2016 2:13:49 PM
src/main/java/gov/gao/epds/controller/HomeController.java java 53 Lines 7.4 KB Sep 12, 2016 12:05:58 PM
src/main/java/gov/gao/epds/controller/ProtestInfoController.java java 340 Lines 36.8 KB Nov 6, 2016 8:01:34 PM
src/main/java/gov/gao/epds/controller/UserInfoController.java java 242 Lines 32.1 KB Oct 30, 2016 8:05:40 PM
src/main/java/gov/gao/epds/dao/Agency_Info_DAO.java java 58 Lines 5 KB Oct 28, 2016 1:56:55 PM
src/main/java/gov/gao/epds/dao/File_Info_DAO.java java 229 Lines 17.3 KB Nov 6, 2016 8:02:27 PM
src/main/java/gov/gao/epds/dao/GC_Track_Service_DAO.java java 59 Lines 4.9 KB Oct 28, 2016 1:57:06 PM
src/main/java/gov/gao/epds/dao/Protest_Info_DAO.java java 574 Lines 50.6 KB Nov 6, 2016 8:01:51 PM
src/main/java/gov/gao/epds/dao/User_Info_DAO.java java 539 Lines 41 KB Nov 6, 2016 8:02:34 PM
src/main/java/gov/gao/epds/dto/AbstractFileInfo.java java 2 Lines 220 bytes Jul 26, 2016 10:17:19 AM
src/main/java/gov/gao/epds/dto/AddAttorneyNote.java java 6 Lines 688 bytes Oct 20, 2016 1:28:45 PM
src/main/java/gov/gao/epds/dto/AdvancedSearchDTO.java java 105 Lines 9.6 KB Oct 20, 2016 2:22:04 PM
src/main/java/gov/gao/epds/dto/BrowserLogs.java java 11 Lines 1.2 KB Mar 16, 2016 5:57:46 PM
src/main/java/gov/gao/epds/dto/CaseCompletionStatus.java java 36 Lines 3.3 KB Sep 14, 2016 10:23:54 AM
src/main/java/gov/gao/epds/dto/CompanyInfo.java java 31 Lines 2.5 KB Oct 10, 2016 2:33:31 PM
src/main/java/gov/gao/epds/dto/DTOValidator.java java 15 Lines 2.8 KB Oct 31, 2016 9:44:50 PM
src/main/java/gov/gao/epds/dto/DashboardPreferences.java java 5 Lines 414 bytes May 4, 2016 9:56:08 AM
src/main/java/gov/gao/epds/dto/EditFileInfo.java java 6 Lines 860 bytes Oct 30, 2016 4:12:59 PM
src/main/java/gov/gao/epds/dto/EditProtestInfo.java java 14 Lines 2 KB Nov 6, 2016 7:43:40 PM
src/main/java/gov/gao/epds/dto/FileStorage.java java 9 Lines 1.4 KB Aug 24, 2016 12:59:31 PM
src/main/java/gov/gao/epds/dto/FileUploadDTO.java java 12 Lines 1.4 KB Aug 1, 2016 10:40:11 AM
src/main/java/gov/gao/epds/dto/FileUploadError.java java 8 Lines 762 bytes Aug 1, 2016 10:24:04 AM

src/main/java/gov/gao/epds/dto/InputValidationError.java java 8 Lines 728 bytes Jul 18, 2016 2:01:10 PM
src/main/java/gov/gao/epds/dto/LoginDTO.java java 8 Lines 1.1 KB Aug 8, 2016 11:24:03 AM
src/main/java/gov/gao/epds/dto/ProtestInfoFormDTO.java java 87 Lines 9.2 KB Oct 19, 2016 9:32:11 AM
src/main/java/gov/gao/epds/dto/Recaptcha.java java 8 Lines 836 bytes Nov 7, 2016 7:55:25 AM
src/main/java/gov/gao/epds/dto/RemoveIntervenor.java java 6 Lines 793 bytes Jul 15, 2016 5:12:20 PM
src/main/java/gov/gao/epds/dto/SubmitNewDocDTO.java java 58 Lines 7.3 KB Jul 15, 2016 4:05:25 PM
src/main/java/gov/gao/epds/dto/TemplateDataDTO.java java 13 Lines 1.4 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/dto/UpdateComments.java java 6 Lines 760 bytes Aug 10, 2016 2:19:52 AM
src/main/java/gov/gao/epds/dto/UpdateTypeOfDoc.java java 6 Lines 878 bytes Nov 6, 2016 7:40:11 PM
src/main/java/gov/gao/epds/dto/UploadedFileIdentifier.java java 5 Lines 697 bytes Mar 7, 2016 1:26:57 PM
src/main/java/gov/gao/epds/dto/User_info_auth.java java 43 Lines 3.5 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/dto/User_info_dto.java java 94 Lines 10.7 KB Aug 22, 2016 9:01:02 AM
src/main/java/gov/gao/epds/dto/ValidateToken.java java 6 Lines 818 bytes Jul 15, 2016 5:09:52 PM
src/main/java/gov/gao/epds/dto/ViewCaseDocketFileInfo.java java 10 Lines 1.3 KB Nov 6, 2016 7:34:33 PM
src/main/java/gov/gao/epds/filestorage/FileStorageCryptoException.java java 3 Lines 400 bytes Aug 16, 2016 4:10:50 PM
src/main/java/gov/gao/epds/filestorage/FileStorageCryptoUtils.java java 96 Lines 7.2 KB Oct 10, 2016 11:10:35 PM
src/main/java/gov/gao/epds/filestorage/FileStorageEncryption.java java 126 Lines 17.9 KB Oct 10, 2016 11:08:28 PM
src/main/java/gov/gao/epds/filestorage/SFTP.java java 137 Lines 7.6 KB Oct 30, 2016 7:22:22 PM
src/main/java/gov/gao/epds/filestorage/ZipUnZipUtils.java java 140 Lines 13.3 KB Oct 10, 2016 11:09:29 PM
src/main/java/gov/gao/epds/fileupload/Utils/ChunkFileInfo.java java 72 Lines 4.7 KB Oct 5, 2016 3:36:08 PM
src/main/java/gov/gao/epds/fileupload/Utils/ChunkFileInfoStorage.java java 29 Lines 2.3 KB Jul 15, 2016 5:13:43 PM
src/main/java/gov/gao/epds/fileupload/Utils/DownloadTask.java java 3 Lines 506 bytes Oct 18, 2016 2:37:19 PM
src/main/java/gov/gao/epds/fileupload/Utils/HttpUtils.java java 14 Lines 774 bytes Feb 10, 2016 3:37:56 PM
src/main/java/gov/gao/epds/fileupload/Utils/ReusableInputStream.java java 20 Lines 1.7 KB Jul 27, 2016 3:30:17 PM
src/main/java/gov/gao/epds/fileupload/Utils/ValidateExcelDocuments.java java 1 Lines 122 bytes Jul 27, 2016 2:12:39 PM
src/main/java/gov/gao/epds/fileupload/Utils/ValidatePdfDocuments.java java 1 Lines 120 bytes Jul 27, 2016 2:12:57 PM
src/main/java/gov/gao/epds/fileupload/Utils/ValidateWordDocuments.java java 1 Lines 121 bytes Jul 27, 2016 2:11:52 PM
src/main/java/gov/gao/epds/fileupload/Utils/ValidateZipDocuments.java java 1 Lines 120 bytes Jul 27, 2016 2:13:10 PM
src/main/java/gov/gao/epds/filters/Cors.java java 16 Lines 2.8 KB Apr 25, 2016 8:40:54 AM
src/main/java/gov/gao/epds/filters/CorsFilter.java java 2.2 KB Apr 25, 2016 1:14:14 PM
src/main/java/gov/gao/epds/filters/CsrfHeaderFilter.java java 17 Lines 4.5 KB Jun 16, 2016 12:58:22 AM
src/main/java/gov/gao/epds/filters/CustomPreAuthenticationFilter.java java 39 Lines 4.2 KB Sep 13, 2016 12:47:24 PM
src/main/java/gov/gao/epds/gctrack/Attorney.java java 7 Lines 526 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/AuthenticationFilter.java java 16 Lines 1.8 KB Sep 26, 2016 4:14:29 PM
src/main/java/gov/gao/epds/gctrack/AuthenticationService.java java 21 Lines 1.7 KB Sep 26, 2016 5:47:25 PM
src/main/java/gov/gao/epds/gctrack/BatchUpdateLogInfo.java java 13 Lines 1.6 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/EPDS_Update.java java 17 Lines 1.7 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/EPDS_event.java java 13 Lines 1.1 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/EpdsUpdate.java java 5 Lines 717 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/Event_Notification_Response.java java 5 Lines 425 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/GCTrackRestApp.java java 4 Lines 538 bytes Sep 26, 2016 5:59:42 PM
src/main/java/gov/gao/epds/gctrack/GCTrackService.java java 163 Lines 14.5 KB Oct 4, 2016 11:11:02 AM
src/main/java/gov/gao/epds/gctrack/GC_Track_Service_Call_Response.java java 9 Lines 727 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/GC_Track_WebService_Event_Test.java java 8 Lines 928 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/GC_test_dataHandler.java java 209 Lines 15.5 KB Jun 8, 2016 12:14:10 PM
src/main/java/gov/gao/epds/gctrack/GC_track_case.java java 45 Lines 3 KB Nov 7, 2016 8:20:30 AM
src/main/java/gov/gao/epds/gctrack/Gc_Track_Update2.java java 7 Lines 558 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/Gc_protest_info.java java 59 Lines 6.2 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/NewEpdsEventsPerCase.java java 17 Lines 1.5 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/NewProtest.java java 11 Lines 939 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/OrgUnit.java java 5 Lines 316 bytes Feb 10, 2016 3:37:57 PM

src/main/java/gov/gao/epds/gctrack/Party.java java 5 Lines 338 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/gctrack/SecurityInterceptor.java java 37 Lines 4 KB Sep 26, 2016 4:55:23 PM
src/main/java/gov/gao/epds/logging/ui/LoggingUtil.java java 2 Lines 679 bytes Jun 13, 2016 8:42:06 AM
src/main/java/gov/gao/epds/persistence/DataAccess.java java 185 Lines 12.6 KB Oct 28, 2016 1:57:24 PM
src/main/java/gov/gao/epds/persistence/entity/Agency_Info.java java 29 Lines 2.3 KB Sep 14, 2016 10:16:35 AM
src/main/java/gov/gao/epds/persistence/entity/CustomTrackingRevisionEntity.java java 36 Lines 3.8 KB Sep 14, 2016 10:17:32 AM
src/main/java/gov/gao/epds/persistence/entity/Doc_Info.java java 11 Lines 1.1 KB Sep 6, 2016 10:45:22 AM
src/main/java/gov/gao/epds/persistence/entity/File_Alert.java java 29 Lines 2.2 KB Sep 14, 2016 10:18:01 AM
src/main/java/gov/gao/epds/persistence/entity/File_Info.java java 82 Lines 8.6 KB Oct 11, 2016 6:13:09 AM
src/main/java/gov/gao/epds/persistence/entity/GAO_User.java java 37 Lines 2.7 KB Sep 14, 2016 10:20:08 AM
src/main/java/gov/gao/epds/persistence/entity/GC_Track_Service_Event.java java 91 Lines 8.8 KB Oct 4, 2016 10:18:03 PM
src/main/java/gov/gao/epds/persistence/entity/Invited_User.java java 80 Lines 8.2 KB Oct 4, 2016 10:18:16 PM
src/main/java/gov/gao/epds/persistence/entity/Protest_Dm_Info.java java 46 Lines 5.5 KB Oct 4, 2016 10:18:34 PM
src/main/java/gov/gao/epds/persistence/entity/Protest_File_Bridge.java java 26 Lines 3 KB Oct 4, 2016 10:18:48 PM
src/main/java/gov/gao/epds/persistence/entity/Protest_Info.java java 372 Lines 37.3 KB Oct 31, 2016 11:35:12 PM
src/main/java/gov/gao/epds/persistence/entity/Tier_1_Agency.java java 25 Lines 2.3 KB Sep 14, 2016 10:25:05 AM
src/main/java/gov/gao/epds/persistence/entity/Tier_2_Agency.java java 29 Lines 2.6 KB Sep 14, 2016 10:25:28 AM
src/main/java/gov/gao/epds/persistence/entity/User_Info.java java 149 Lines 12.3 KB Sep 14, 2016 11:53:41 AM
src/main/java/gov/gao/epds/persistence/entity/User_Protest_Role_Bridge.java java 41 Lines 5.4 KB Oct 3, 2016 5:30:53 PM
src/main/java/gov/gao/epds/persistence/entity/User_Role.java java 6 Lines 720 bytes Sep 14, 2016 11:58:24 AM
src/main/java/gov/gao/epds/persistence/interceptors/AuditLogInterceptor.java java 1.9 KB Sep 7, 2016 1:52:04 PM
src/main/java/gov/gao/epds/persistence/listener/CustomEntityTrackingRevisionListener.java java 17 Lines 2 KB Sep 7, 2016 3:23:24 PM
src/main/java/gov/gao/epds/rest/auth/services/AuthResourceProvider.java java 20 Lines 1.4 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/rest/auth/services/AuthUtil.java java 84 Lines 7.6 KB Oct 30, 2016 3:22:44 PM
src/main/java/gov/gao/epds/rest/auth/services/AuthenticationController.java java 222 Lines 21.9 KB Oct 12, 2016 8:39:14 AM
src/main/java/gov/gao/epds/rest/auth/services/ServiceResponse.java java 14 Lines 1.1 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/security/CustomAccessDeniedHandler.java java 9 Lines 1.3 KB Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/security/PreAuthenticationException.java java 2 Lines 288 bytes Apr 14, 2016 2:31:43 PM
src/main/java/gov/gao/epds/security/RestAuthenticationEntryPoint.java java 12 Lines 1.4 KB Jun 15, 2016 5:43:29 PM
src/main/java/gov/gao/epds/service/CaseDocketSheetService.java java 477 Lines 39.6 KB Nov 6, 2016 8:02:49 PM
src/main/java/gov/gao/epds/service/CustomUserDetailsService.java java 5 Lines 1.3 KB Feb 10, 2016 3:37:55 PM
src/main/java/gov/gao/epds/service/DashboardService.java java 405 Lines 27.6 KB Nov 6, 2016 8:02:57 PM
src/main/java/gov/gao/epds/service/EmailService.java java 199 Lines 16.7 KB Nov 6, 2016 8:03:04 PM
src/main/java/gov/gao/epds/service/GC_Service.java java 351 Lines 33.9 KB Oct 19, 2016 1:32:29 PM
src/main/java/gov/gao/epds/service/HomeService.java java 237 Lines 25.3 KB Nov 6, 2016 8:03:12 PM
src/main/java/gov/gao/epds/service/ProtestInfoService.java java 468 Lines 45.2 KB Nov 6, 2016 8:01:34 PM
src/main/java/gov/gao/epds/service/UserInfoService.java java 300 Lines 32 KB Nov 1, 2016 9:25:05 AM
src/main/java/gov/gao/epds/service/recaptcha/RecaptchaController.java java 13 Lines 2.1 KB Nov 7, 2016 7:56:01 AM
src/main/java/gov/gao/epds/service/recaptcha/RecaptchaService.java java 176 bytes Jun 8, 2016 3:08:52 PM
src/main/java/gov/gao/epds/service/recaptcha/RecaptchaServiceException.java java 2 Lines 318 bytes Jun 8, 2016 3:08:58 PM
src/main/java/gov/gao/epds/service/recaptcha/RecaptchaServiceImpl.java java 21 Lines 2.8 KB Nov 7, 2016 7:56:22 AM
src/main/java/gov/gao/epds/service/recaptcha/RestTemplateConfig.java java 6 Lines 1.2 KB Jun 8, 2016 3:09:09 PM
src/main/java/gov/gao/epds/session/EpdsSession.java java 24 Lines 2.4 KB Jul 27, 2016 10:04:57 AM
src/main/java/gov/gao/epds/session/GlobalAttribute.java java 1 Lines 156 bytes Feb 10, 2016 3:37:57 PM
src/main/java/gov/gao/epds/test/MainClass.java java 21 Lines 2.9 KB Oct 31, 2016 1:19:48 AM
src/main/java/gov/gao/epds/test/ServiceTest.java java 98 Lines 6.7 KB Jun 15, 2016 5:47:22 PM
src/main/java/gov/gao/epds/test/TestDataHandler.java java 20 Lines 2.4 KB Feb 10, 2016 3:37:54 PM
src/main/java/gov/gao/epds/test/UploadExecutor.java java 9 Lines 623 bytes Oct 18, 2016 10:03:16 AM
src/main/java/gov/gao/epds/test/UploadThread.java java 3 Lines 390 bytes Oct 18, 2016 10:03:59 AM
src/main/java/gov/gao/epds/test/ValidationTesting.java java 20 Lines 2.7 KB Oct 30, 2016 5:34:52 PM

src/main/java/gov/gao/epds/testutils/EmailValidatorTest.java java 7 Lines 606 bytes Jul 7, 2016 4:12:33 PM

src/main/java/gov/gao/epds/testutils/Util.java java 35 Lines 3.3 KB Oct 27, 2016 10:58:09 AM

src/main/java/gov/gao/epds/utills/Date_Util.java java 167 Lines 11.2 KB Oct 5, 2016 2:57:00 PM

src/main/java/gov/gao/epds/utills/EPDS_FileUtils.java java 185 Lines 18 KB Nov 6, 2016 8:03:30 PM

src/main/java/gov/gao/epds/utills/ExportToExcel.java java 135 Lines 10.8 KB Jun 15, 2016 5:50:32 PM

src/main/java/gov/gao/epds/utills/GC_track_util.java java 131 Lines 13 KB Oct 6, 2016 10:38:06 AM

src/main/java/gov/gao/epds/utills/GlobalFields.java java 7 Lines 882 bytes Mar 7, 2016 1:21:51 PM

src/main/java/gov/gao/epds/utills/GlobalParams.java java 26 Lines 2.4 KB Sep 1, 2016 10:42:35 AM

src/main/java/gov/gao/epds/utills/HibernateUtil.java java 10 Lines 983 bytes Sep 7, 2016 4:27:21 PM

src/main/java/gov/gao/epds/utills/Params.java java 17 Lines 1.2 KB Jun 14, 2016 3:55:35 AM

src/main/java/gov/gao/epds/utills/PayDotGovUtil.java java 35 Lines 3.7 KB Oct 31, 2016 8:38:00 AM

src/main/java/gov/gao/epds/utills/PreAuthUtil.java java 69 Lines 7.8 KB Nov 6, 2016 7:55:37 PM

src/main/java/gov/gao/epds/utills/PropertyFileEncrypter.java java 18 Lines 1.6 KB Nov 7, 2016 9:29:12 AM

src/main/java/gov/gao/epds/utills/Protest_info_util.java java 221 Lines 24.2 KB Nov 6, 2016 8:01:34 PM

src/main/java/gov/gao/epds/utills/PublicHoliday.java java 211 Lines 11.8 KB Oct 5, 2016 2:52:46 PM

src/main/java/gov/gao/epds/utills/PublicHoliday_Util.java java 1 Lines 73 bytes Feb 10, 2016 3:37:57 PM

src/main/java/gov/gao/epds/utills/RegistrationUtil.java java 223 Lines 13.5 KB Nov 6, 2016 8:01:34 PM

src/main/java/gov/gao/epds/utills/RegularExpressions.java java 3 Lines 204 bytes Jul 5, 2016 1:36:26 PM

src/main/java/gov/gao/epds/utills/RemoveUnlimitedJCERestriction.java java 25 Lines 3.5 KB Sep 15, 2016 3:27:19 PM

src/main/java/gov/gao/epds/utills/SpringApplicationContext.java java 4 Lines 727 bytes Feb 10, 2016 3:37:57 PM

src/main/java/gov/gao/epds/utills/Util.java java 189 Lines 12.5 KB Oct 30, 2016 8:49:37 AM

src/main/java/gov/gao/epds/utills/Util_Test.java java 28 Lines 2.9 KB Feb 10, 2016 3:37:55 PM

src/main/java/gov/gao/epds/utills/ZipFile_Util.java java 108 Lines 11.3 KB Oct 11, 2016 5:41:19 AM

src/main/java/gov/gao/epds/utills/templates/HtmlToPDF.java java 165 Lines 12.9 KB Oct 10, 2016 1:21:06 PM

src/main/java/gov/gao/epds/utills/templates/TemplateUtils.java java 147 Lines 9.4 KB Oct 10, 2016 1:32:25 PM

src/main/java/gov/treas/fms/services/tcsonline/Classification.java java 4 Lines 2.1 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/ClassificationData.java java 5 Lines 2.3 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/CompleteOnlineCollection.java java 4 Lines 1.9 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/CompleteOnlineCollectionRequest.java java 6 Lines 2.2 KB Feb 10, 2016 3:37:57 PM

src/main/java/gov/treas/fms/services/tcsonline/CompleteOnlineCollectionResponse.java java 4 Lines 1.6 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/CompleteOnlineCollectionResponseType.java java 4 Lines 1.9 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/CreateForce.java java 4 Lines 1.6 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/CreateForceRequest.java java 12 Lines 4.6 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/CreateForceResponse.java java 4 Lines 1.6 KB Feb 10, 2016 3:37:57 PM

src/main/java/gov/treas/fms/services/tcsonline/CreateForceResponseType.java java 4 Lines 1.6 KB Feb 10, 2016 3:37:57 PM

src/main/java/gov/treas/fms/services/tcsonline/CustomCollectionFields.java java 26 Lines 9.5 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/GetDetails.java java 4 Lines 1.6 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/GetDetailsRequest.java java 8 Lines 2.9 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/GetDetailsResponse.java java 4 Lines 1.6 KB Feb 10, 2016 3:37:57 PM

src/main/java/gov/treas/fms/services/tcsonline/GetDetailsResponseType.java java 4 Lines 1.6 KB Feb 10, 2016 3:37:57 PM

src/main/java/gov/treas/fms/services/tcsonline/ObjectFactory.java java 41 Lines 11 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/StartOnlineCollection.java java 4 Lines 1.8 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/StartOnlineCollectionRequest.java java 36 Lines 13.3 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/StartOnlineCollectionResponse.java java 4 Lines 1.4 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/StartOnlineCollectionResponseType.java java 4 Lines 1.8 KB Feb 10, 2016 3:37:57 PM

src/main/java/gov/treas/fms/services/tcsonline/TCSOnlineService.java java 2 Lines 4.2 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tcsonline/TCSOnlineService_Service.java java 21 Lines 3.1 KB Feb 10, 2016 3:37:57 PM

src/main/java/gov/treas/fms/services/tconline/TCSServiceFault.java java 6 Lines 2.2 KB Feb 10, 2016 3:37:57 PM

src/main/java/gov/treas/fms/services/tconline/TCSServiceFault_Exception.java java 5 Lines 1.2 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tconline/TransactionData.java java 16 Lines 6.4 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tconline/TransactionType.java java 9 Lines 1.2 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tconline/Transactions.java java 4 Lines 1.9 KB Feb 10, 2016 3:37:58 PM

src/main/java/gov/treas/fms/services/tconline/package-info.java java 136 bytes Feb 10, 2016 3:37:58 PM

src/main/resources/copyOfWorkinglog4j.properties java_properties 877 bytes Jun 15, 2016 11:28:00 AM

src/main/resources/epds.local.properties java_properties 3.6 KB Sep 8, 2016 1:02:35 PM

src/main/resources/epds.oireasd0108.properties java_properties 3.5 KB Sep 12, 2016 9:25:31 PM

src/main/resources/epds.oireasd0110.properties java_properties 3.7 KB Sep 12, 2016 9:24:51 PM

src/main/resources/file-storage-keystore.properties java_properties 488 bytes Sep 1, 2016 10:30:48 AM

src/main/resources/log4j.properties java_properties 877 bytes Oct 2, 2016 3:14:46 PM

src/main/webapp/WEB-INF/spring/dispatcher-servlet.xml xml 5.7 KB Oct 3, 2016 5:36:09 PM

src/main/webapp/WEB-INF/spring/spring-security.xml xml 2.9 KB Jun 14, 2016 8:54:43 AM

src/main/webapp/WEB-INF/web.xml xml 2.8 KB Sep 7, 2016 12:21:19 PM

src/main/webapp/resources/GAO_Templates/html/ACKNOWLEDGMENTLETTERSWITHOUTPROTECTIVEORDER.htm
html 14.7 KB Oct 10, 2016 11:52:22 AM

src/main/webapp/resources/GAO_Templates/html/ACKNOWLEDGMENTLETTERSWITHPROTECTIVEORDER.htm html 41.8
KB Oct 10, 2016 7:00:26 PM

src/main/webapp/resources/GAO_Templates/html/ACKNOWLEDGMENTOFCLAIMFORCOSTS.htm html 11.8 KB Mar 24,
2016 3:53:20 PM

src/main/webapp/resources/GAO_Templates/html/ADMISSIONTOPROTECTIVEORDER.htm html 9.7 KB Oct 10, 2016 7:42:58
PM

src/main/webapp/resources/GAO_Templates/html/AMENDEDADMISSIONTOPROTECTIVEORDER.htm html 9.8 KB Oct 10,
2016 7:43:21 PM

src/main/webapp/resources/GAO_Templates/html/ENTITLEMENTACKNOWLEDGMENTOFFREQUEST.htm html 12.2 KB Mar
24, 2016 3:53:20 PM

src/main/webapp/resources/GAO_Templates/html/NOTICE.htm html 4.8 KB Mar 24, 2016 3:53:20 PM

src/main/webapp/resources/GAO_Templates/html/PROTECTIVEORDERNOTICEAPPLICATION.htm html 38.3 KB Oct 10,
2016 2:03:49 PM

src/main/webapp/resources/GAO_Templates/html/WITHDRAWALCONFIRMATION.htm html 5.1 KB Mar 24, 2016 3:53:20 PM

src/main/webapp/resources/Zip_File_Contents/data/filters/angular-filter.min.js javascript 1 Lines 19.6 KB Feb 10, 2016 3:37:59 PM

src/main/webapp/resources/Zip_File_Contents/data/script.js javascript 61 Lines 4.7 KB Apr 25, 2016 1:02:05 PM

src/main/webapp/resources/Zip_File_Contents/data/scripts/angular-cookies.js javascript 40 Lines 5.7 KB Feb 10, 2016 3:37:59 PM

src/main/webapp/resources/Zip_File_Contents/data/scripts/angular-local-storage.js javascript 206 Lines 14.9 KB Feb 10, 2016
3:37:59 PM

src/main/webapp/resources/Zip_File_Contents/data/vendor/angular/angular-route.js javascript 205 Lines 34.9 KB Feb 10, 2016
3:37:59 PM

src/main/webapp/resources/Zip_File_Contents/data/vendor/angular/angular.js javascript 6,198 Lines 938.1 KB Feb 10, 2016 3:37:59
PM

src/main/webapp/resources/Zip_File_Contents/data/vendor/bootstrap/js/bootstrap.js javascript 1,292 Lines 67.3 KB Feb 10, 2016
3:37:59 PM

src/main/webapp/resources/Zip_File_Contents/data/vendor/bootstrap/js/bootstrap.min.js javascript 2 Lines 36 KB Feb 10, 2016
3:38:01 PM

src/main/webapp/resources/Zip_File_Contents/data/vendor/bootstrap/js/npm.js javascript 12 Lines 484 bytes Feb 10, 2016 3:37:59
PM

src/main/webapp/resources/Zip_File_Contents/data/views/case-docket-file-info-view.html html 1 Lines 4.5 KB May 18, 2016
1:12:22 AM

src/main/webapp/resources/Zip_File_Contents/index.html html 1 Lines 6.9 KB Jun 10, 2016 12:07:41 PM

src/main/webapp/resources/emailTemplates/add-edit-notes.tpl.htm html 1.1 KB Aug 9, 2016 10:24:21 AM

src/main/webapp/resources/emailTemplates/caseAccessDenied.tpl.html html 1.1 KB Oct 30, 2016 7:51:14 PM

src/main/webapp/resources/emailTemplates/caseDocketFilings.tpl.html html 1003 bytes Aug 9, 2016 10:28:09 AM
 src/main/webapp/resources/emailTemplates/invitation.tpl.htm html 1.2 KB May 15, 2016 5:28:24 PM
 src/main/webapp/resources/emailTemplates/newProtest.tpl.html html 1.4 KB Aug 9, 2016 10:53:00 AM
 src/main/webapp/resources/emailTemplates/otherProtest.tpl.html html 1.3 KB Aug 9, 2016 10:56:04 AM
 src/main/webapp/resources/emailTemplates/paymentConfirmation.tpl.html html 1.2 KB Aug 9, 2016 10:25:52 AM
 src/main/webapp/resources/template-edit/filename.htm html 8.2 KB Mar 24, 2016 3:53:20 PM
 src/test/java/com/EPDS/US/Dog.java java 19 Lines 1.3 KB Feb 10, 2016 3:37:54 PM
 src/test/java/com/EPDS/US/Student.java java 22 Lines 1.9 KB Feb 10, 2016 3:37:54 PM
 src/test/java/com/EPDS/US/Test.java java 73 Lines 4.5 KB Feb 10, 2016 3:37:54 PM
 src/test/java/com/EPDS/US/TestSendMail.java java 3 Lines 503 bytes Apr 7, 2016 2:08:51 PM
 src/test/java/gov/gao/edps/webService/GcTrackCase2.java java 15 Lines 1.3 KB Feb 10, 2016 3:37:54 PM
 src/test/java/gov/gao/edps/webService/RestClientConfig.java java 17 Lines 2.5 KB Feb 10, 2016 3:37:54 PM
 src/test/java/gov/gao/edps/webService/SimpleClient.java java 810 bytes Feb 10, 2016 3:37:54 PM
 src/test/java/gov/gao/edps/webService/Test.java java 187 Lines 14.8 KB Oct 7, 2016 11:22:09 AM
 src/test/java/gov/gao/edps/clam/test/ArbitraryInputStream.java java 17 Lines 1.2 KB Apr 1, 2016 4:34:11 PM
 src/test/java/gov/gao/edps/clam/test/ArbitraryInputStreamTestCase.java java 26 Lines 2 KB Apr 1, 2016 4:34:11 PM
 src/test/java/gov/gao/edps/clam/test/ClamScanTestCase.java java 37 Lines 4.2 KB Apr 1, 2016 4:38:48 PM
 src/test/java/gov/gao/edps/filestorage/test/FileEncryptionTest.java java 2 Lines 1009 bytes Aug 22, 2016 9:00:44 AM
 src/test/java/gov/usda/edps/test/RemoveMapElement.java java 5 Lines 1.1 KB Aug 26, 2016 3:00:38 PM
 src/test/java/gov/usda/edps/test/RemoveMapElementCacheBuilder.java java 14 Lines 1.2 KB Aug 26, 2016 2:58:34 PM
 src/test/java/gov/usda/edps/test/Sha512Test.java java 16 Lines 1.1 KB May 13, 2016 4:34:52 PM
 src/test/java/gov/usda/edps/test/Test.java java 143 Lines 11.8 KB Oct 5, 2016 3:32:16 PM
 target/classes/copyOfWorkinglog4j.properties java_properties 877 bytes Nov 6, 2016 7:53:23 PM
 target/classes/edps.local.properties java_properties 3.6 KB Nov 6, 2016 7:53:23 PM
 target/classes/edps.oireasd0108.properties java_properties 3.5 KB Nov 6, 2016 7:53:23 PM
 target/classes/edps.oireasd0110.properties java_properties 3.7 KB Nov 6, 2016 7:53:23 PM
 target/classes/file-storage-keystore.properties java_properties 488 bytes Nov 6, 2016 7:53:23 PM
 target/classes/log4j.properties java_properties 877 bytes Nov 6, 2016 7:53:23 PM
 target/m2e-wtp/web-resources/META-INF/maven/goa.gao.epds/SpringSecurityEPDS/pom.properties java_properties 260 bytes
 Nov 7, 2016 10:39:04 AM
 target/m2e-wtp/web-resources/META-INF/maven/goa.gao.epds/SpringSecurityEPDS/pom.xml xml 13.9 KB Nov 7, 2016 10:39:04 AM

Reference Elements

Classpath:

C:\Users\mhussaini\.m2\repository\antlr\antlr\2.7.7\antlr-2.7.7.jar
 C:\Users\mhussaini\.m2\repository\aoapalliance\aoapalliance\1.0\aoapalliance-1.0.jar
 C:\Users\mhussaini\.m2\repository\ch\qos\logback\logback-classic\1.1.3\logback-classic-1.1.3.jar
 C:\Users\mhussaini\.m2\repository\ch\qos\logback\logback-core\1.1.3\logback-core-1.1.3.jar
 C:\Users\mhussaini\.m2\repository\com\fastxml\classmate\1.3.0\classmate-1.3.0.jar
 C:\Users\mhussaini\.m2\repository\com\fastxml\jackson\core\jackson-annotations\2.5.0\jackson-annotations-2.5.0.jar
 C:\Users\mhussaini\.m2\repository\com\fastxml\jackson\core\jackson-core\2.5.3\jackson-core-2.5.3.jar
 C:\Users\mhussaini\.m2\repository\com\fastxml\jackson\core\jackson-databind\2.5.3\jackson-databind-2.5.3.jar
 C:\Users\mhussaini\.m2\repository\com\github\virtuald\curvesapi\1.03\curvesapi-1.03.jar
 C:\Users\mhussaini\.m2\repository\com\google\code\gson\gson\2.5\gson-2.5.jar
 C:\Users\mhussaini\.m2\repository\com\google\guava\guava\19.0\guava-19.0.jar
 C:\Users\mhussaini\.m2\repository\com\googlecode\json-simple\json-simple\1.1\json-simple-1.1.jar
 C:\Users\mhussaini\.m2\repository\com\itextpdf\itextpdf\5.5.8\itextpdf-5.5.8.jar
 C:\Users\mhussaini\.m2\repository\com\itextpdf\tool\xmlworker\5.5.8\xmlworker-5.5.8.jar

C:\Users\mhussaini\.m2\repository\com\jcraft\jsch\0.1.52\jsch-0.1.52.jar
C:\Users\mhussaini\.m2\repository\com\nimbusds\nimbus-jose-jwt\4.7\nimbus-jose-jwt-4.7.jar
C:\Users\mhussaini\.m2\repository\com\sun\istack\istack-commons-runtime\2.16\istack-commons-runtime-2.16.jar
C:\Users\mhussaini\.m2\repository\com\sun\xml\bind\jaxb-core\2.2.7\jaxb-core-2.2.7.jar
C:\Users\mhussaini\.m2\repository\com\sun\xml\bind\jaxb-impl\2.2.7\jaxb-impl-2.2.7.jar
C:\Users\mhussaini\.m2\repository\com\sun\xml\fastinfoset\FastInfoset\1.2.12\FastInfoset-1.2.12.jar
C:\Users\mhussaini\.m2\repository\commons-beanutils\commons-beanutils\1.9.2\commons-beanutils-1.9.2.jar
C:\Users\mhussaini\.m2\repository\commons-codec\commons-codec\1.9\commons-codec-1.9.jar
C:\Users\mhussaini\.m2\repository\commons-collections\commons-collections\3.2.2\commons-collections-3.2.2.jar
C:\Users\mhussaini\.m2\repository\commons-digester\commons-digester\1.8.1\commons-digester-1.8.1.jar
C:\Users\mhussaini\.m2\repository\commons-fileupload\commons-fileupload\1.3.1\commons-fileupload-1.3.1.jar
C:\Users\mhussaini\.m2\repository\commons-io\commons-io\2.5\commons-io-2.5.jar
C:\Users\mhussaini\.m2\repository\commons-lang\commons-lang\2.6\commons-lang-2.6.jar
C:\Users\mhussaini\.m2\repository\commons-logging\commons-logging\1.2\commons-logging-1.2.jar
C:\Users\mhussaini\.m2\repository\commons-validator\commons-validator\1.5.1\commons-validator-1.5.1.jar
C:\Users\mhussaini\.m2\repository\de\idyl\winzipaes\1.0.1\winzipaes-1.0.1.jar
C:\Users\mhussaini\.m2\repository\dom4j\dom4j\1.6.1\dom4j-1.6.1.jar
C:\Users\mhussaini\.m2\repository\javax\activation\activation\1.1.1\activation-1.1.1.jar
C:\Users\mhussaini\.m2\repository\javax\annotation\jsr250-api\1.0\jsr250-api-1.0.jar
C:\Users\mhussaini\.m2\repository\javax\el\el-api\2.2\el-api-2.2.jar
C:\Users\mhussaini\.m2\repository\javax\enterprise\cdi-api\1.1\cdi-api-1.1.jar
C:\Users\mhussaini\.m2\repository\javax\inject\javax.inject\1\javax.inject-1.jar
C:\Users\mhussaini\.m2\repository\javax\mail\javax.mail-api\1.5.4\javax.mail-api-1.5.4.jar
C:\Users\mhussaini\.m2\repository\javax\servlet\javax.servlet-api\3.1.0\javax.servlet-api-3.1.0.jar
C:\Users\mhussaini\.m2\repository\javax\servlet\jsp\javax.servlet.jsp-api\2.3.1\javax.servlet.jsp-api-2.3.1.jar
C:\Users\mhussaini\.m2\repository\javax\servlet\jstl\1.2\jstl-1.2.jar
C:\Users\mhussaini\.m2\repository\javax\transaction\jta\1.1\jta-1.1.jar
C:\Users\mhussaini\.m2\repository\javax\validation\validation-api\1.1.0.Final\validation-api-1.1.0.Final.jar
C:\Users\mhussaini\.m2\repository\javax\xml\bind\jaxb-api\2.2.7\jaxb-api-2.2.7.jar
C:\Users\mhussaini\.m2\repository\javax\xml\bind\jsr173_api\1.0\jsr173_api-1.0.jar
C:\Users\mhussaini\.m2\repository\joda-time\joda-time\2.9.3\joda-time-2.9.3.jar
C:\Users\mhussaini\.m2\repository\log4j\apache-log4j-extras\1.2.17\apache-log4j-extras-1.2.17.jar
C:\Users\mhussaini\.m2\repository\log4j\log4j\1.2.17\log4j-1.2.17.jar
C:\Users\mhussaini\.m2\repository\mysql\mysql-connector-java\5.1.31\mysql-connector-java-5.1.31.jar
C:\Users\mhussaini\.m2\repository\net\jcip\jcip-annotations\1.0\jcip-annotations-1.0.jar
C:\Users\mhussaini\.m2\repository\net\lingala\zip4j\zip4j\1.3.2\zip4j-1.3.2.jar
C:\Users\mhussaini\.m2\repository\net\minidev\json-smart\1.3.1\json-smart-1.3.1.jar
C:\Users\mhussaini\.m2\repository\net\sf\sevenzipjbinding\sevenzipjbinding\9.20-2.00beta\sevenzipjbinding-9.20-2.00beta.jar
C:\Users\mhussaini\.m2\repository\net\sourceforge\jexcelapi\jxl\2.6.12\jxl-2.6.12.jar
C:\Users\mhussaini\.m2\repository\org\apache\geronimo\specs\geronimo-jta_1.1_spec\1.1.1\geronimo-jta_1.1_spec-1.1.1.jar
C:\Users\mhussaini\.m2\repository\org\apache\httpcomponents\httpclient\4.5.2\httpclient-4.5.2.jar
C:\Users\mhussaini\.m2\repository\org\apache\httpcomponents\httpcore\4.4.4\httpcore-4.4.4.jar
C:\Users\mhussaini\.m2\repository\org\apache\poi\poi-ooxml-schemas\3.14\poi-ooxml-schemas-3.14.jar
C:\Users\mhussaini\.m2\repository\org\apache\poi\poi-ooxml\3.14\poi-ooxml-3.14.jar
C:\Users\mhussaini\.m2\repository\org\apache\poi\poi\3.14\poi-3.14.jar
C:\Users\mhussaini\.m2\repository\org\apache\xmlbeans\xmlbeans\2.6.0\xmlbeans-2.6.0.jar
C:\Users\mhussaini\.m2\repository\org\aspectj\aspectjrt\1.8.1\aspectjrt-1.8.1.jar
C:\Users\mhussaini\.m2\repository\org\bouncycastle\bcprov-ext-jdk15on\1.53\bcprov-ext-jdk15on-1.53.jar
C:\Users\mhussaini\.m2\repository\org\bouncycastle\bcprov-jdk15on\1.53\bcprov-jdk15on-1.53.jar
C:\Users\mhussaini\.m2\repository\org\bouncycastle\bcprov-jdk16\1.45\bcprov-jdk16-1.45.jar
C:\Users\mhussaini\.m2\repository\org\codehaus\jackson\jackson-core-asl\1.9.12\jackson-core-asl-1.9.12.jar

C:\Users\mhussaini\.m2\repository\org\codehaus\jackson\jackson-jaxrs\1.9.12\jackson-jaxrs-1.9.12.jar
C:\Users\mhussaini\.m2\repository\org\codehaus\jackson\jackson-mapper-asl\1.9.12\jackson-mapper-asl-1.9.12.jar
C:\Users\mhussaini\.m2\repository\org\codehaus\jackson\jackson-xc\1.9.12\jackson-xc-1.9.12.jar
C:\Users\mhussaini\.m2\repository\org\codehaus\jettison\jettison\1.3.2\jettison-1.3.2.jar
C:\Users\mhussaini\.m2\repository\org\hibernate\common\hibernate-commons-annotations\5.0.1.Final\hibernate-commons-annotations-5.0.1.Final.jar
C:\Users\mhussaini\.m2\repository\org\hibernate\hibernate-core\5.2.2.Final\hibernate-core-5.2.2.Final.jar
C:\Users\mhussaini\.m2\repository\org\hibernate\hibernate-envers\5.2.2.Final\hibernate-envers-5.2.2.Final.jar
C:\Users\mhussaini\.m2\repository\org\hibernate\hibernate-validator\5.3.0.CR1\hibernate-validator-5.3.0.CR1.jar
C:\Users\mhussaini\.m2\repository\org\hibernate\javax\persistence\hibernate-jpa-2.1-api\1.0.0.Final\hibernate-jpa-2.1-api-1.0.0.Final.jar
C:\Users\mhussaini\.m2\repository\org\jasypt\jasypt\1.9.2\jasypt-1.9.2.jar
C:\Users\mhussaini\.m2\repository\org\javassist\javassist\3.20.0-GA\javassist-3.20.0-GA.jar
C:\Users\mhussaini\.m2\repository\org\jboss\jandex\2.0.0.Final\jandex-2.0.0.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\logging\jboss-logging\3.3.0.Final\jboss-logging-3.3.0.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\resteasy\async-http-servlet-3.0\3.0.19.Final\async-http-servlet-3.0-3.0.19.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\resteasy\resteasy-client\3.0.13.Final\resteasy-client-3.0.13.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\resteasy\resteasy-jackson-provider\3.0.13.Final\resteasy-jackson-provider-3.0.13.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\resteasy\resteasy-jaxb-provider\3.0.13.Final\resteasy-jaxb-provider-3.0.13.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\resteasy\resteasy-jaxrs\3.0.13.Final\resteasy-jaxrs-3.0.13.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\resteasy\resteasy-jettison-provider\3.0.13.Final\resteasy-jettison-provider-3.0.13.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\resteasy\resteasy-servlet-initializer\3.0.19.Final\resteasy-servlet-initializer-3.0.19.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\resteasy\resteasy-spring\3.0.13.Final\resteasy-spring-3.0.13.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\spec\javax\annotation\jboss-annotations-api_1.2_spec\1.0.0.Final\jboss-annotations-api_1.2_spec-1.0.0.Final.jar
C:\Users\mhussaini\.m2\repository\org\jboss\spec\javax\interceptor\jboss-interceptors-api_1.1_spec\1.0.0.Beta1\jboss-interceptors-api_1.1_spec-1.0.0.Beta1.jar
C:\Users\mhussaini\.m2\repository\org\jboss\spec\javax\ws\rs\jboss-jaxrs-api_2.0_spec\1.0.0.Final\jboss-jaxrs-api_2.0_spec-1.0.0.Final.jar
C:\Users\mhussaini\.m2\repository\org\jsoup\jsoup\1.8.3\jsoup-1.8.3.jar
C:\Users\mhussaini\.m2\repository\org\slf4j\jcl-over-slf4j\1.7.12\jcl-over-slf4j-1.7.12.jar
C:\Users\mhussaini\.m2\repository\org\slf4j\slf4j-api\1.7.12\slf4j-api-1.7.12.jar
C:\Users\mhussaini\.m2\repository\org\slf4j\slf4j-log4j12\1.7.12\slf4j-log4j12-1.7.12.jar
C:\Users\mhussaini\.m2\repository\org\springframework\security\spring-security-config\4.1.3.RELEASE\spring-security-config-4.1.3.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\security\spring-security-core\4.1.3.RELEASE\spring-security-core-4.1.3.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\security\spring-security-web\4.1.3.RELEASE\spring-security-web-4.1.3.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-aop\4.3.2.RELEASE\spring-aop-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-beans\4.3.2.RELEASE\spring-beans-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-context\4.3.2.RELEASE\spring-context-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-core\4.3.2.RELEASE\spring-core-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-expression\4.3.2.RELEASE\spring-expression-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-jdbc\4.3.2.RELEASE\spring-jdbc-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-orm\4.3.2.RELEASE\spring-orm-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-tx\4.3.2.RELEASE\spring-tx-4.3.2.RELEASE.jar

C:\Users\mhussaini\.m2\repository\org\springframework\spring-web\4.3.2.RELEASE\spring-web-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\org\springframework\spring-webmvc\4.3.2.RELEASE\spring-webmvc-4.3.2.RELEASE.jar
C:\Users\mhussaini\.m2\repository\stax\stax-api\1.0.1\stax-api-1.0.1.jar
C:\Users\mhussaini\Downloads\ojdbc7.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\activation\api\main\activation-1.1.1.redhat-5.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\annotation\api\main\jboss-annotations-api_1.2_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\batch\api\main\jboss-batch-api_1.0_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\ejb\api\main\jboss-ejb-api_3.2_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\el\api\main\jboss-el-api_3.0_spec-1.0.6.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\enterprise\api\main\cdi-api-1.2.0.redhat-2.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\enterprise\concurrent\api\main\jboss-concurrency-api_1.0_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\faces\api\main\jboss-jsf-api_2.2_spec-2.2.12.redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\inject\api\main\javax.inject-1.0.0.redhat-6.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\interceptor\api\main\jboss-interceptors-api_1.2_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\jms\api\main\jboss-jms-api_2.0_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\jws\api\main\jsr181-api-1.0.0.MR1-redhat-8.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\mail\api\main\javax.mail-1.5.5.redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\management\j2ee\api\main\jboss-j2eemgmt-api_1.1_spec-1.0.1.Final-redhat-4.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\persistence\api\main\hibernate-jpa-2.1-api-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\resource\api\main\jboss-connector-api_1.7_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\security\auth\message\api\main\jboss-jaspi-api_1.1_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\security\jacc\api\main\jboss-jacc-api_1.5_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\servlet\api\main\jboss-servlet-api_3.1_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\servlet\jsp\api\main\jboss-jsp-api_2.3_spec-1.0.1.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\servlet\jstl\api\main\jboss-jstl-api_1.2_spec-1.1.2.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\transaction\api\main\jboss-transaction-api_1.2_spec-1.0.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\validation\api\main\validation-api-1.1.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\websocket\api\main\jboss-websocket-api_1.1_spec-1.1.1.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\ws\rs\api\main\jboss-jaxrs-api_2.0_spec-1.0.0.Final-redhat-1.jar

C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\wsdl4j\api\main\wsdl4j-1.6.3.redhat-2.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\xml\bind\api\main\jboss-jaxb-api_2.2_spec-1.0.4.Final-redhat-4.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\xml\rpc\api\main\jboss-jaxrpc-api_1.1_spec-1.0.1.Final-redhat-5.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\xml\soap\api\main\jboss-saaj-api_1.3_spec-1.0.3.Final-redhat-2.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\javax\xml\ws\api\main\jboss-jaxws-api_2.2_spec-2.0.2.Final-redhat-2.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\hibernate\validator\main\hibernate-validator-5.2.4.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\as\controller-client\main\wildfly-controller-client-2.1.2.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\dmr\main\jboss-dmr-1.3.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\ejb3\main\jboss-ejb3-ext-api-2.2.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\logging\main\jboss-logging-3.3.0.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\resteasy\resteasy-jaxb-provider\main\resteasy-jaxb-provider-3.0.16.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\resteasy\resteasy-jaxrs\main\async-http-servlet-3.0-3.0.16.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\resteasy\resteasy-jaxrs\main\resteasy-client-3.0.16.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\resteasy\resteasy-jaxrs\main\resteasy-jaxrs-3.0.16.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\jboss\resteasy\resteasy-multipart-provider\main\resteasy-multipart-provider-3.0.16.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\picketbox\main\picketbox-4.9.6.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\picketbox\main\picketbox-commons-1.0.0.final-redhat-4.jar
C:\Users\mhussaini\devstudio\runtimes\jboss-eap\modules\system\layers\base\org\picketbox\main\picketbox-infinispan-4.9.6.Final-redhat-1.jar
C:\Users\mhussaini\devstudio\studio\plugins\org.hamcrest.core_1.3.0.v201303031735.jar
C:\Users\mhussaini\devstudio\studio\plugins\org.junit_4.12.0.v201504281640\junit.jar

Libdirs:

No libdirs specified during translation

Rulepacks

Valid Rulepacks:

Name: HPE Security Fortify Secure Coding Rules, Core, ABAP

Version: 2016.1.0.0008

ID: 5A663288-B1F7-416A-AEED-E5F00DC3596F

SKU: RUL13095

Name: Fortify Secure Coding Rules, Core Preview, ABAP
Version: 2014.4.0.0008
ID: 5249BACE-3D49-4937-BFFF-48BA414E14DD
SKU: RUL13187

Name: HPE Security Fortify Secure Coding Rules, Core, ActionScript 3.0
Version: 2016.1.0.0008
ID: 92127AA2-E666-4F28-B1C1-C0F6A939A089
SKU: RUL13094

Name: HPE Security Fortify Secure Coding Rules, Core, Android
Version: 2016.1.0.0008
ID: FF9890E6-D119-4EE8-A591-83DCF4CA6952
SKU: RUL13093

Name: HPE Security Fortify Secure Coding Rules, Core, Annotations
Version: 2016.1.0.0008
ID: 14EE50EB-FA1C-4AE8-8B59-39F952E21E3B
SKU: RUL13078

Name: HPE Security Fortify Secure Coding Rules, Core, ColdFusion
Version: 2016.1.0.0008
ID: EEA7C678-058E-462A-8A59-AF925F7B7164
SKU: RUL13024

Name: HPE Security Fortify Secure Coding Rules, Core, COBOL
Version: 2016.1.0.0008
ID: 9EB21F2C-ED22-440B-82A3-E38DC2533F03
SKU: RUL13088

Name: HPE Security Fortify Secure Coding Rules, Core, C/C++
Version: 2016.1.0.0008
ID: 711E0652-7494-42BE-94B1-DB3799418C7E
SKU: RUL13001

Name: HPE Security Fortify Secure Coding Rules, Core, .NET
Version: 2016.1.0.0008
ID: D57210E5-E762-4112-97DD-019E61D32D0E
SKU: RUL13002

Name: HPE Security Fortify Secure Coding Rules, Core, Java
Version: 2016.1.0.0008
ID: 06A6CC97-8C3F-4E73-9093-3E74C64A2AAF
SKU: RUL13003

Name: HPE Security Fortify Secure Coding Rules, Core, JavaScript
Version: 2016.1.0.0008
ID: BD292C4E-4216-4DB8-96C7-9B607BFD9584
SKU: RUL13059

Name: HPE Security Fortify Secure Coding Rules, Core, Objective-C
Version: 2016.1.0.0008
ID: B18E58BA-8B2D-4FC5-83BF-378594CAD260
SKU: RUL13099

Name: HPE Security Fortify Secure Coding Rules, Core, PHP
Version: 2016.1.0.0008
ID: 343CBB32-087C-4A4E-8BD8-273B5F876069
SKU: RUL13058

Name: HPE Security Fortify Secure Coding Rules, Core, Python
Version: 2016.1.0.0008
ID: FD15CBE4-E059-4CBB-914E-546BDCEB422B
SKU: RUL13083

Name: HPE Security Fortify Secure Coding Rules, Core, Ruby
Version: 2016.1.0.0008
ID: CA8013D5-11DE-44EF-9563-182F9FCB87BC
SKU: RUL13135

Name: HPE Security Fortify Secure Coding Rules, Core, SQL
Version: 2016.1.0.0008
ID: 6494160B-E1DB-41F5-9840-2B1609EE7649
SKU: RUL13004

Name: HPE Security Fortify Secure Coding Rules, Core, Swift
Version: 2016.1.0.0008
ID: A977528A-B2C7-412D-9691-F1668E05154B
SKU: RUL13147

Name: HPE Security Fortify Secure Coding Rules, Core, Classic ASP, VBScript, and VB6
Version: 2016.1.0.0008
ID: 1D426B6F-8D33-4AD6-BBCE-237ABAFAB924
SKU: RUL13060

Name: HPE Security Fortify Secure Coding Rules, Extended, Configuration
Version: 2016.1.0.0008
ID: CD6959FC-0C37-45BE-9637-BAA43C3A4D56
SKU: RUL13005

Name: HPE Security Fortify Secure Coding Rules, Extended, Content
Version: 2016.1.0.0008
ID: 9C48678C-09B6-474D-B86D-97EE94D38F17
SKU: RUL13067

Name: HPE Security Fortify Secure Coding Rules, Extended, C/C++
Version: 2016.1.0.0008
ID: BD4641AD-A6FF-4401-A8F4-6873272F2748
SKU: RUL13006

Name: HPE Security Fortify Secure Coding Rules, Extended, .NET

Version: 2016.1.0.0008
ID: 557BCC56-CD42-43A7-B4FE-CDD00D58577E
SKU: RUL13027

Name: HPE Security Fortify Secure Coding Rules, Extended, Java
Version: 2016.1.0.0008
ID: AAAC0B10-79E7-4FE5-9921-F4903A79D317
SKU: RUL13007

Name: HPE Security Fortify Secure Coding Rules, Extended, JavaScript
Version: 2016.1.0.0008
ID: C4D1969E-B734-47D3-87D4-73962C1D32E2
SKU: RUL13141

Name: HPE Security Fortify Secure Coding Rules, Extended, JSP
Version: 2016.1.0.0008
ID: 00403342-15D0-48C9-8E67-4B1CFBDEFCD2
SKU: RUL13026

Name: HPE Security Fortify Secure Coding Rules, Extended, SQL
Version: 2016.1.0.0008
ID: 4BC5B2FA-C209-4DBC-9C3E-1D3EEFAF135A
SKU: RUL13025

External Metadata:

Name: CWE
ID: 3ADB9EE4-5761-4289-8BD3-CBFCC593EBBC

Name: FISMA
ID: B40F9EE0-3824-4879-B9FE-7A789C89307C

Name: NIST SP 800-53 Rev.4
ID: 1114583B-EA24-45BE-B7F8-B61201BACDD0

Name: OWASP Mobile 2014
ID: EEE3F9E7-28D6-4456-8761-3DA56C36F4EE

Name: OWASP Top 10 2004
ID: 771C470C-9274-4580-8556-C023E4D3ADB4

Name: OWASP Top 10 2007
ID: 1EB1EC0E-74E6-49A0-BCE5-E6603802987A

Name: OWASP Top 10 2010
ID: FDCECA5E-C2A8-4BE8-BB26-76A8ECD0ED59

Name: OWASP Top 10 2013
ID: 1A2B4C7E-93B0-4502-878A-9BE40D2A25C4

Name: PCI 1.1

ID: CBDB9D4D-FC20-4C04-AD58-575901CAB531

Name: PCI 1.2

ID: 57940BDB-99F0-48BF-BF2E-CFC42BA035E5

Name: PCI 2.0

ID: 8970556D-7F9F-4EA7-8033-9DF39D68FF3E

Name: PCI 3.0

ID: E2FB0D38-0192-4F03-8E01-FE2A12680CA3

Name: PCI 3.1

ID: AC0D18CF-C1DA-47CF-9F1A-E8EC0A4A717E

Name: SANS Top 25 2009

ID: 939EF193-507A-44E2-ABB7-C00B2168B6D8

Name: SANS Top 25 2010

ID: 72688795-4F7B-484C-88A6-D4757A6121CA

Name: SANS Top 25 2011

ID: 92EB4481-1FD9-4165-8E16-F2DE6CB0BD63

Name: STIG 3.1

ID: F2FA57EA-5AAA-4DDE-90A5-480BE65CE7E7

Name: STIG 3.10

ID: 788A87FE-C9F9-4533-9095-0379A9B35B12

Name: STIG 3.4

ID: 58E2C21D-C70F-4314-8994-B859E24CF855

Name: STIG 3.5

ID: DD18E81F-3507-41FA-9DFA-2A9A15B5479F

Name: STIG 3.6

ID: 000CA760-0FED-4374-8AA2-6FA3968A07B1

Name: STIG 3.7

ID: E69C07C0-81D8-4B04-9233-F3E74167C3D2

Name: STIG 3.9

ID: 1A9D736B-2D4A-49D1-88CA-DF464B40D732

Name: WASC 2.00

ID: 74f8081d-dd49-49da-880f-6830cebe9777

Name: WASC 24 + 2

ID: 9DC61E7F-1A48-4711-BBFD-E9DFF537871F

Properties

WinForms.CollectionMutationMonitor.Label=WinFormsDataSource
WinForms.ExtractEventHandlers=true
WinForms.TransformChangeNotificationPattern=true
WinForms.TransformDataBindings=true
WinForms.TransformMessageLoops=true
awt.toolkit=sun.awt.windows.WToolkit
com.fortify.AuthenticationKey=C:\Users\mhussaini\AppData\Local\Fortify/config/tools
com.fortify.Core=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\Core
com.fortify.InstallRoot=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30
com.fortify.InstallationUserName=MHussaini
com.fortify.SCAExecutablePath=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\bin\sourceanalyzer.exe
com.fortify.TotalPhysicalMemory=16971313152
com.fortify.VS.DefaultFramework.10.0=DotNetFrameworkv40
com.fortify.VS.DefaultFramework.11.0=DotNetFrameworkv45
com.fortify.VS.DefaultFramework.12.0=DotNetFrameworkv451
com.fortify.VS.DefaultFramework.14.0=DotNetFrameworkv451
com.fortify.VS.Frameworks.DotNetFrameworkv11.SDKRegistry="SOFTWARE\Microsoft\NETFramework", "sdkInstallRootv1.1", "bin"
com.fortify.VS.Frameworks.DotNetFrameworkv11.VSProjectVersionCode=1;
com.fortify.VS.Frameworks.DotNetFrameworkv20.SDKRegistry="SOFTWARE\Microsoft\NETFramework", "sdkInstallRootv2.0", "bin"
com.fortify.VS.Frameworks.DotNetFrameworkv20.VSProjectVersionCode=131072
com.fortify.VS.Frameworks.DotNetFrameworkv30.SDKRegistry="SOFTWARE\Microsoft\Microsoft SDKs\Windows\v6.0A", "InstallationFolder"
com.fortify.VS.Frameworks.DotNetFrameworkv30.VSProjectVersionCode=196608
com.fortify.VS.Frameworks.DotNetFrameworkv35.SDKRegistry="SOFTWARE\Microsoft\Microsoft SDKs\Windows\v6.0A", "InstallationFolder"
com.fortify.VS.Frameworks.DotNetFrameworkv35.VSProjectVersionCode=196613
com.fortify.VS.Frameworks.DotNetFrameworkv40.DotNetFolder="v4.0.30319"
com.fortify.VS.Frameworks.DotNetFrameworkv40.SDKRegistry="SOFTWARE\Microsoft\Microsoft SDKs\Windows\v7.0A\WinSDK-NetFx40Tools-x86", "InstallationFolder"
com.fortify.VS.Frameworks.DotNetFrameworkv40.VSProjectVersionCode=262144
com.fortify.VS.Frameworks.DotNetFrameworkv45.DotNetFolder="v4.0.30319"
com.fortify.VS.Frameworks.DotNetFrameworkv45.SDKPaths="Microsoft SDKs\Windows\v8.0A\bin\NETFX 4.0 Tools\"; "Microsoft SDKs\Windows\v8.0A\bin\"; "Microsoft SDKs\Windows\v8.0\bin\NETFX 4.0 Tools\"; "Microsoft SDKs\Windows\v8.0\bin\
com.fortify.VS.Frameworks.DotNetFrameworkv45.SDKRegistry="SOFTWARE\Microsoft\Microsoft SDKs\Windows\v8.0A\WinSDK-NetFx40Tools-x86", "InstallationFolder"
com.fortify.VS.Frameworks.DotNetFrameworkv45.VSProjectVersionCode=262149
com.fortify.VS.Frameworks.DotNetFrameworkv451.DotNetFolder="v4.0.30319"
com.fortify.VS.Frameworks.DotNetFrameworkv451.SDKPaths="Microsoft SDKs\Windows\v8.1A\bin\NETFX 4.5.1 Tools\"; "Microsoft SDKs\Windows\v8.1A\bin\"; "Microsoft SDKs\Windows\v8.1\bin\NETFX 4.5.1 Tools\"; "Microsoft SDKs\Windows\v8.1\bin\
com.fortify.VS.Frameworks.DotNetFrameworkv451.SDKRegistry="SOFTWARE\Microsoft\Microsoft SDKs\Windows\v8.1A\WinSDK-NetFx40Tools-x86", "InstallationFolder"
com.fortify.VS.Frameworks.DotNetFrameworkv451.VSProjectVersionCode=451
com.fortify.VS.Frameworks.default.SDKRegistry="SOFTWARE\Microsoft\Microsoft SDKs\Windows\v8.0A\WinSDK-NetFx35Tools-x86", "InstallationFolder"; "SOFTWARE\Microsoft\Microsoft SDKs\Windows\v7.0A\WinSDK-NetFx35Tools-

```
x86","InstallationFolder";"SOFTWARE\Microsoft\Microsoft SDKs\Windows\v6.1","InstallationFolder","\bin"
com.fortify.VS.Frameworks.default.VSProjectVersionCode=-1
com.fortify.VS.RequireASPPrecompilation=true
com.fortify.WorkingDirectory=C:\Users\mhussaini\AppData\Local\Fortify
com.fortify.locale=en
com.fortify.sca.AddImpliedMethods=true
com.fortify.sca.AntCompilerClass=com.fortify.dev.ant.SCACompiler
com.fortify.sca.BuildID=SpringSecurityEPDS
com.fortify.sca.BundleControlflowIssues=true
com.fortify.sca.BytecodePreview=true
com.fortify.sca.CollectPerformanceData=true
com.fortify.sca.CustomRulesDir=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\Core\config\customrules
com.fortify.sca.DaemonCompilers=com.fortify.sca.util.compilers.GppCompiler,com.fortify.sca.util.compilers.GccCompiler,com.f
ortify.sca.util.compilers.AppleGppCompiler,com.fortify.sca.util.compilers.AppleGccCompiler,com.fortify.sca.util.compilers.Micr
osoftCompiler,com.fortify.sca.util.compilers.MicrosoftLinker,com.fortify.sca.util.compilers.LdCompiler,com.fortify.sca.util.com
pilers.ArUtil,com.fortify.sca.util.compilers.SunCCompiler,com.fortify.sca.util.compilers.SunCppCompiler,com.fortify.sca.util.co
mpilers.IntelCompiler,com.fortify.sca.util.compilers.ExternalCppAdapter,com.fortify.sca.util.compilers.ClangCompiler
com.fortify.sca.DeadCodeFilter=true
com.fortify.sca.DeadCodeIgnoreTrivialPredicates=true
com.fortify.sca.DefaultAnalyzers=semantic:dataflow:controlflow:nullptr:configuration:content:structural:buffer
com.fortify.sca.DefaultFileTypes=java,rb,jsp,jsp,tag,tagx,tld,sql,cfm,php,phtml,ctp,pks,pkh,pkb,xml,config,settings,properties,dl
l,exe,inc,asp,vbscript,js,ini,bas,cls,vbs,frm,ctl,html,htm,xsd,wsdd,xmi,py,cfml,cfc,abap,xhtml,cpx,xcfg,jsff,as,mxml,cbl,cscfg,csde
f,wadcfg,appxmanifest,wsdl,plist
com.fortify.sca.DefaultJarsDirs=default_jars
com.fortify.sca.DefaultRulesDir=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\Core\config\rules
com.fortify.sca.DisableDeadCodeElimination=false
com.fortify.sca.DisableFunctionPointers=false
com.fortify.sca.DisableGlobals=false
com.fortify.sca.DisableInferredConstants=false
com.fortify.sca.DisplayProgress=true
com.fortify.sca.EnableInterproceduralConstantResolution=true
com.fortify.sca.EnableNestedWrappers=true
com.fortify.sca.EnableStructuralMatchCache=true
com.fortify.sca.EnableWrapperDetection=true
com.fortify.sca.FVDLDisableDescriptions=false
com.fortify.sca.FVDLDisableProgramData=false
com.fortify.sca.FVDLDisableSnippets=false
com.fortify.sca.FVDLStylesheet=C:\Program
Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\Core/resources/sca/fvdl2html.xsl
com.fortify.sca.IndirectCallGraphBuilders=WinFormsAdHocFunctionBuilder,VirtualCGBuilder,J2EEIndirectCGBuilder,JNICG
Builder,StoredProcedureResolver,JavaWSCGBuilder,StrutsCGBuilder,DotNetWSCGBuilder,SqlServerSPResolver,ASPCGBuild
er,ScriptedCGBuilder,NewJspCustomTagCGBuilder,DotNetCABCGBuilder,StateInjectionCGBuilder,SqlServerSPResolver2,PH
PLambdaResolver,JavaWebCGBuilder
com.fortify.sca.JVMArgs=-Djava.awt.headless=true -Dcom.sun.management.jmxremote=true -
XX:SoftRefLRUPolicyMSPerMB=3000 -Xss16M -Xmx1800M -Xms300M -server
com.fortify.sca.JavaNewBeta=true
com.fortify.sca.JdkVersion=1.5
com.fortify.sca.LowSeverityCutoff=1.0
com.fortify.sca.MachineOutputMode=
com.fortify.sca.NoNestedOutTagOutput=org.apache.taglibs.standard.tag.rt.core.RemoveTag,org.apache.taglibs.standard.tag.rt.cor
e.SetTag
```

```
com.fortify.sca.PID=14164
com.fortify.sca.Phase0HigherOrder.Languages=python,ruby
com.fortify.sca.Phase0HigherOrder.Level=1
com.fortify.sca.Phase0HigherOrder.Limiter.Values=16
com.fortify.sca.PidFile=C:\Users\MHUSSA~1\AppData\Local\Temp\1\PID6220745697811485703.tmp
com.fortify.sca.PrintPerformanceDataAfterScan=false
com.fortify.sca.ProjectRoot=C:\Users\mhussaini\AppData\Local\Fortify
com.fortify.sca.ProjectRoot=C:\Users\mhussaini\AppData\Local\Fortify
com.fortify.sca.Renderer=fpr
com.fortify.sca.RequireMapKeys=classrule
com.fortify.sca.ResultsFile=C:\AmersFiles\Gitworkspace\SpringSecurityEPDS\SpringSecurityEPDS_scan.fpr
com.fortify.sca.SolverTimeout=15
com.fortify.sca.SqlLanguage=TSQL
com.fortify.sca.SuppressLowSeverity=true
com.fortify.sca.ThreadCount.NameTableLoading=1
com.fortify.sca.ThreadCount.PreTransform=1
com.fortify.sca.TypeInferenceFunctionTimeout=60
com.fortify.sca.TypeInferenceLanguages=javascript,python,ruby
com.fortify.sca.TypeInferencePhase0Timeout=300
com.fortify.sca.UnicodeInputFile=true
com.fortify.sca.UniversalBlacklist=.*yyparse.*
com.fortify.sca.WrapperHeuristic=moderate
com.fortify.sca.analyzer.controlflow.EnableLivenessOptimization=false
com.fortify.sca.analyzer.controlflow.EnableMachineFiltering=false
com.fortify.sca.analyzer.controlflow.EnableRefRuleOptimization=false
com.fortify.sca.analyzer.controlflow.EnableTimeOut=true
com.fortify.sca.compilers.ant=com.fortify.sca.util.compilers.AntAdapter
com.fortify.sca.compilers.ar=com.fortify.sca.util.compilers.ArUtil
com.fortify.sca.compilers.armcc=com.fortify.sca.util.compilers.ArmCcCompiler
com.fortify.sca.compilers.armcpp=com.fortify.sca.util.compilers.ArmCppCompiler
com.fortify.sca.compilers.c++=com.fortify.sca.util.compilers.GppCompiler
com.fortify.sca.compilers.c89=com.fortify.sca.util.compilers.C89Compiler
com.fortify.sca.compilers.cc=com.fortify.sca.util.compilers.GccCompiler
com.fortify.sca.compilers.cl=com.fortify.sca.util.compilers.MicrosoftCompiler
com.fortify.sca.compilers.clearmake=com.fortify.sca.util.compilers.TouchlessCompiler
com.fortify.sca.compilers.devenv=com.fortify.sca.util.compilers.DevenvNetAdapter
com.fortify.sca.compilers.fortify=com.fortify.sca.util.compilers.FortifyCompiler
com.fortify.sca.compilers.g++=com.fortify.sca.util.compilers.GppCompiler
com.fortify.sca.compilers.g++-*=com.fortify.sca.util.compilers.GppCompiler
com.fortify.sca.compilers.g++2*=com.fortify.sca.util.compilers.GppCompiler
com.fortify.sca.compilers.g++3*=com.fortify.sca.util.compilers.GppCompiler
com.fortify.sca.compilers.g++4*=com.fortify.sca.util.compilers.GppCompiler
com.fortify.sca.compilers.gcc=com.fortify.sca.util.compilers.GccCompiler
com.fortify.sca.compilers.gcc-*=com.fortify.sca.util.compilers.GccCompiler
com.fortify.sca.compilers.gcc2*=com.fortify.sca.util.compilers.GccCompiler
com.fortify.sca.compilers.gcc3*=com.fortify.sca.util.compilers.GccCompiler
com.fortify.sca.compilers.gcc4*=com.fortify.sca.util.compilers.GccCompiler
com.fortify.sca.compilers.gmake=com.fortify.sca.util.compilers.TouchlessCompiler
com.fortify.sca.compilers.icc=com.fortify.sca.util.compilers.IntelCompiler
com.fortify.sca.compilers.icl=com.fortify.sca.util.compilers.MicrosoftCompiler
com.fortify.sca.compilers.icpc=com.fortify.sca.util.compilers.IntelCompiler
```

com.fortify.sca.compilers.jam=com.fortify.sca.util.compilers.TouchlessCompiler
com.fortify.sca.compilers.javac=com.fortify.sca.util.compilers.JavacCompiler
com.fortify.sca.compilers.ld=com.fortify.sca.util.compilers.LdCompiler
com.fortify.sca.compilers.link=com.fortify.sca.util.compilers.MicrosoftLinker
com.fortify.sca.compilers.make=com.fortify.sca.util.compilers.TouchlessCompiler
com.fortify.sca.compilers.msbuild=com.fortify.sca.util.compilers.MSBuildAdapter
com.fortify.sca.compilers.msdev=com.fortify.sca.util.compilers.DevenvAdapter
com.fortify.sca.compilers.mvn=com.fortify.sca.util.compilers.MavenAdapter
com.fortify.sca.compilers.nmake=com.fortify.sca.util.compilers.TouchlessCompiler
com.fortify.sca.compilers.tcc=com.fortify.sca.util.compilers.ArmCcCompiler
com.fortify.sca.compilers.tcpp=com.fortify.sca.util.compilers.ArmCppCompiler
com.fortify.sca.compilers.touchless=com.fortify.sca.util.compilers.FortifyCompiler
com.fortify.sca.compilers.xilink=com.fortify.sca.util.compilers.MicrosoftLinker
com.fortify.sca.cpfe.441.command=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\Core/private-bin/sca/cpfe441.rfct
com.fortify.sca.cpfe.command=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\Core/private-bin/sca/cpfe48.exe
com.fortify.sca.cpfe.file.option=--gen_c_file_name
com.fortify.sca.cpfe.options=--remove_unneeded_entities --suppress_vtbl -tused
com.fortify.sca.cpfe.options=--remove_unneeded_entities --suppress_vtbl -tused
com.fortify.sca.env.exesearchpath=C:\Users\mhussaini\devstudio\studio;C:\Program Files\CollabNet\Subversion Client;C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\bin;C:\ProgramData\Oracle\Java\javapath;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\WINDOWS\System32\WindowsPowerShell\v1.0;C:\Program Files (x86)\ActivIdentity\ActivClient\;C:\Program Files\ActivIdentity\ActivClient\;C:\WINDOWS\System32\WindowsPowerShell\v1.0;C:\Users\mhussaini\apache-maven-3.3.9\bin;C:\Users\mhussaini\nodejs\;C:\Program Files (x86)\PuTTY\;C:\Program Files (x86)\Riverbed\Steelhead Mobile\;C:\Users\mhussaini\AppData\Local\atom\bin;C:\Users\mhussaini\AppData\Roaming\npm;C:\Users\mhussaini\AppData\Local\VanDyke Software\Clients\
com.fortify.sca.fileextensions.ABAP=ABAP
com.fortify.sca.fileextensions.abap=ABAP
com.fortify.sca.fileextensions.appxmanifest=XML
com.fortify.sca.fileextensions.as=ACTIONSCRIPT
com.fortify.sca.fileextensions.asp=ASP
com.fortify.sca.fileextensions.bas=VB6
com.fortify.sca.fileextensions.cfc=CFML
com.fortify.sca.fileextensions.cfm=CFML
com.fortify.sca.fileextensions.cfml=CFML
com.fortify.sca.fileextensions.cls=VB6
com.fortify.sca.fileextensions.config=XML
com.fortify.sca.fileextensions.cpx=XML
com.fortify.sca.fileextensions.cs=CSHARP
com.fortify.sca.fileextensions.cscfg=XML
com.fortify.sca.fileextensions.csdef=XML
com.fortify.sca.fileextensions.ctl=VB6
com.fortify.sca.fileextensions.ctp=PHP
com.fortify.sca.fileextensions.dll=MSIL
com.fortify.sca.fileextensions.erb=RUBY_ERB
com.fortify.sca.fileextensions.exe=MSIL
com.fortify.sca.fileextensions.faces=JSPX
com.fortify.sca.fileextensions.frm=VB6
com.fortify.sca.fileextensions.htm=HTML

com.fortify.sca.fileextensions.html=HTML
com.fortify.sca.fileextensions.ini=JAVA_PROPERTIES
com.fortify.sca.fileextensions.java=JAVA
com.fortify.sca.fileextensions.js=JAVASCRIPT
com.fortify.sca.fileextensions.jsff=JSPX
com.fortify.sca.fileextensions.jsp=JSP
com.fortify.sca.fileextensions.jspx=JSPX
com.fortify.sca.fileextensions.mdl=MSIL
com.fortify.sca.fileextensions.mod=MSIL
com.fortify.sca.fileextensions.mxml=MXML
com.fortify.sca.fileextensions.php=PHP
com.fortify.sca.fileextensions.phtml=PHP
com.fortify.sca.fileextensions.pkb=PLSQL
com.fortify.sca.fileextensions.pkh=PLSQL
com.fortify.sca.fileextensions.pks=PLSQL
com.fortify.sca.fileextensions.plist=XML
com.fortify.sca.fileextensions.properties=JAVA_PROPERTIES
com.fortify.sca.fileextensions.py=PYTHON
com.fortify.sca.fileextensions.rb=RUBY
com.fortify.sca.fileextensions.settings=XML
com.fortify.sca.fileextensions.sql=SQL
com.fortify.sca.fileextensions.tag=JSP
com.fortify.sca.fileextensions.tagx=JSP
com.fortify.sca.fileextensions.tld=TLID
com.fortify.sca.fileextensions.vb=VB
com.fortify.sca.fileextensions.vbs=VBSCRIPT
com.fortify.sca.fileextensions.vbscript=VBSCRIPT
com.fortify.sca.fileextensions.wadcfg=XML
com.fortify.sca.fileextensions.wsdd=XML
com.fortify.sca.fileextensions.wsdl=XML
com.fortify.sca.fileextensions.xcfg=XML
com.fortify.sca.fileextensions.xhtml=JSPX
com.fortify.sca.fileextensions.xmi=XML
com.fortify.sca.fileextensions.xml=XML
com.fortify.sca.fileextensions.xsd=XML
com.fortify.sca.jsp.UseNativeParser=true
com.fortify.sca.parser.python.ignore.module.1=test.badsyntax_future3
com.fortify.sca.parser.python.ignore.module.2=test.badsyntax_future4
com.fortify.sca.parser.python.ignore.module.3=test.badsyntax_future5
com.fortify.sca.parser.python.ignore.module.4=test.badsyntax_future6
com.fortify.sca.parser.python.ignore.module.5=test.badsyntax_future7
com.fortify.sca.parser.python.ignore.module.6=test.badsyntax_future8
com.fortify.sca.parser.python.ignore.module.7=test.badsyntax_future9
com.fortify.sca.parser.python.ignore.module.8=test.badsyntax_nocaret
com.fortify.search.defaultSyntaxVer=2
com.sun.management.jmxremote=true
dotnet.install.dir=C:\Windows\Microsoft.NET\Framework64
dotnet.v30.referenceAssemblies=C:\Program Files\Reference Assemblies\Microsoft\Framework\v3.0
dotnet.v35.referenceAssemblies=C:\Program Files\Reference Assemblies\Microsoft\Framework\v3.5
file.encoding=Cp1252
file.encoding.pkg=sun.io


```
file.separator=\n
java.awt.graphicsenv=sun.awt.Win32GraphicsEnvironment
java.awt.headless=true
java.awt.printerjob=sun.awt.windows.WPrinterJob
java.class.path=C:\\Program Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\Core\\lib\\exe\\sca-exe.jar
java.class.version=51.0
java.endorsed.dirs=C:\\Program Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\jre\\lib\\endorsed
java.ext.dirs=C:\\Program Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\jre\\lib\\ext;C:\\WINDOWS\\Sun\\Java\\lib\\ext
java.home=C:\\Program Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\jre
java.io.tmpdir=C:\\Users\\MHUSSA~1\\AppData\\Local\\Temp\\1\\
java.library.path=C:\\Program
Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\jre\\bin;C:\\WINDOWS\\Sun\\Java\\bin;C:\\WINDOWS\\system32;C:\\WINDOW
S;C:\\Program Files\\CollabNet\\Subversion Client;C:\\Program
Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\bin;C:\\ProgramData\\Oracle\\Java\\javapath;C:\\WINDOWS\\system32;C:\\WIN
DOWS;C:\\WINDOWS\\System32\\Wbem;C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0;C:\\Program Files
(x86)\\ActivIdentity\\ActivClient;C:\\Program
Files\\ActivIdentity\\ActivClient;C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0;C:\\Users\\mhussaini\\apache-maven-
3.3.9\\bin;C:\\Users\\mhussaini\\nodejs;C:\\Program Files (x86)\\PuTTY;C:\\Program Files (x86)\\Riverbed\\Steelhead
Mobile;C:\\Users\\mhussaini\\AppData\\Local\\atom\\bin;C:\\Users\\mhussaini\\AppData\\Roaming\\npm;C:\\Users\\mhussaini\\AppData\\
Local\\VanDyke Software\\Clients;.
java.rmi.server.randomIDs=true
java.runtime.name=Java(TM) SE Runtime Environment
java.runtime.version=1.7.0_45-b18
java.specification.name=Java Platform API Specification
java.specification.vendor=Oracle Corporation
java.specification.version=1.7
java.vendor=Oracle Corporation
java.vendor.url=http://java.oracle.com/
java.vendor.url.bug=http://bugreport.sun.com/bugreport/
java.version=1.7.0_45
java.vm.info=mixed mode
java.vm.name=Java HotSpot(TM) 64-Bit Server VM
java.vm.specification.name=Java Virtual Machine Specification
java.vm.specification.vendor=Oracle Corporation
java.vm.specification.version=1.7
java.vm.vendor=Oracle Corporation
java.vm.version=24.45-b08
line.separator=

max.file.path.length=255
os.arch=amd64
os.name=Windows 7
os.version=6.1
path.separator=;
stderr.isatty=false
stdout.isatty=false
sun.arch.data.model=64
sun.boot.class.path=C:\\Program Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\jre\\lib\\resources.jar;C:\\Program
Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\jre\\lib\\rt.jar;C:\\Program
Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\jre\\lib\\sunrsasn.jar;C:\\Program
Files\\HP_Fortify\\HP_Fortify_SCA_and_Apps_4.30\\jre\\lib\\jsse.jar;C:\\Program
```

Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\jre\lib\jce.jar;C:\Program
Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\jre\lib\charsets.jar;C:\Program
Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\jre\lib\jfr.jar;C:\Program
Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\jre\classes
sun.boot.library.path=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\jre\bin
sun.cpu.endian=little
sun.cpu.isalist=amd64
sun.desktop=windows
sun.io.unicode.encoding=UnicodeLittle
sun.java.command=C:\Program Files\HP_Fortify\HP_Fortify_SCA_and_Apps_4.30\Core\lib\exe\sca-exe.jar -scan -pid-file
C:\Users\MHUSSA~1\AppData\Local\Temp\1\PID6220745697811485703.tmp
@C:\Users\mhussaini\AppData\Local\Fortify\Eclipse.Plugin-4.30\SpringSecurityEPDS\SpringSecurityEPDSScan.txt
sun.java.launcher=SUN_STANDARD
sun.jnu.encoding=Cp1252
sun.management.compiler=HotSpot 64-Bit Tiered Compilers
sun.os.patch.level=Service Pack 1
user.country=US
user.dir=C:\Users\mhussaini\devstudio\studio
user.home=C:\Users\mhussaini
user.language=en
user.name=MHussaini
user.script=
user.timezone=America/Denver
user.variant=
win32.LocalAppdata=C:\Users\mhussaini\AppData\Local

Commandline Arguments

-scan
-pid-file
C:\Users\MHUSSA~1\AppData\Local\Temp\1\PID6220745697811485703.tmp
-b
SpringSecurityEPDS
-machine-output
-format
fpr
-f
C:\AmersFiles\Gitworkspace\SpringSecurityEPDS\SpringSecurityEPDS_scan.fpr

Warnings

[1237] The following references to java classes could not be resolved. Please make sure to supply all the required jar files that contain these classes to SCA.

Arquillian
ArquillianResource
Deployment
Drone
FindBy
FindByJQuery
InSequence
JSONObject
Root

RunAsClient
WebArchive
WebDriver
WebElement

[236] Your license does not allow access to Fortify SCA for Python

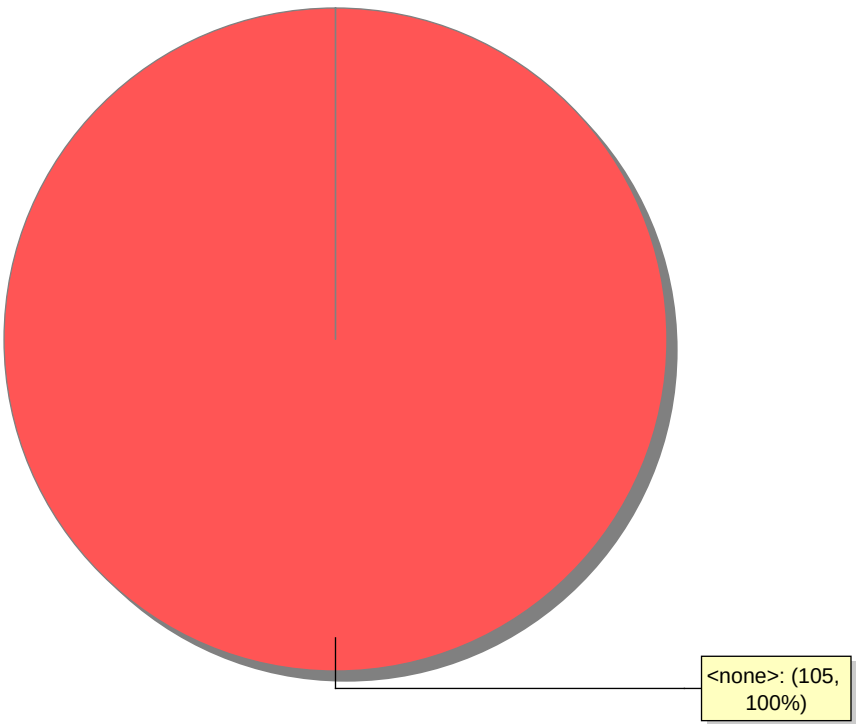
Issue Count by Category

Issues by Category

Password Management: Password in Configuration File	32
Path Manipulation	28
Mass Assignment: Insecure Binder Configuration	25
Mass Assignment: Sensitive Field Exposure	7
JSON Injection	5
Log Forging	4
Cross-Site Scripting: Reflected	2
Password Management: Hardcoded Password	1
Privacy Violation: Heap Inspection	1

Issue Breakdown by Analysis

Issues by Analysis

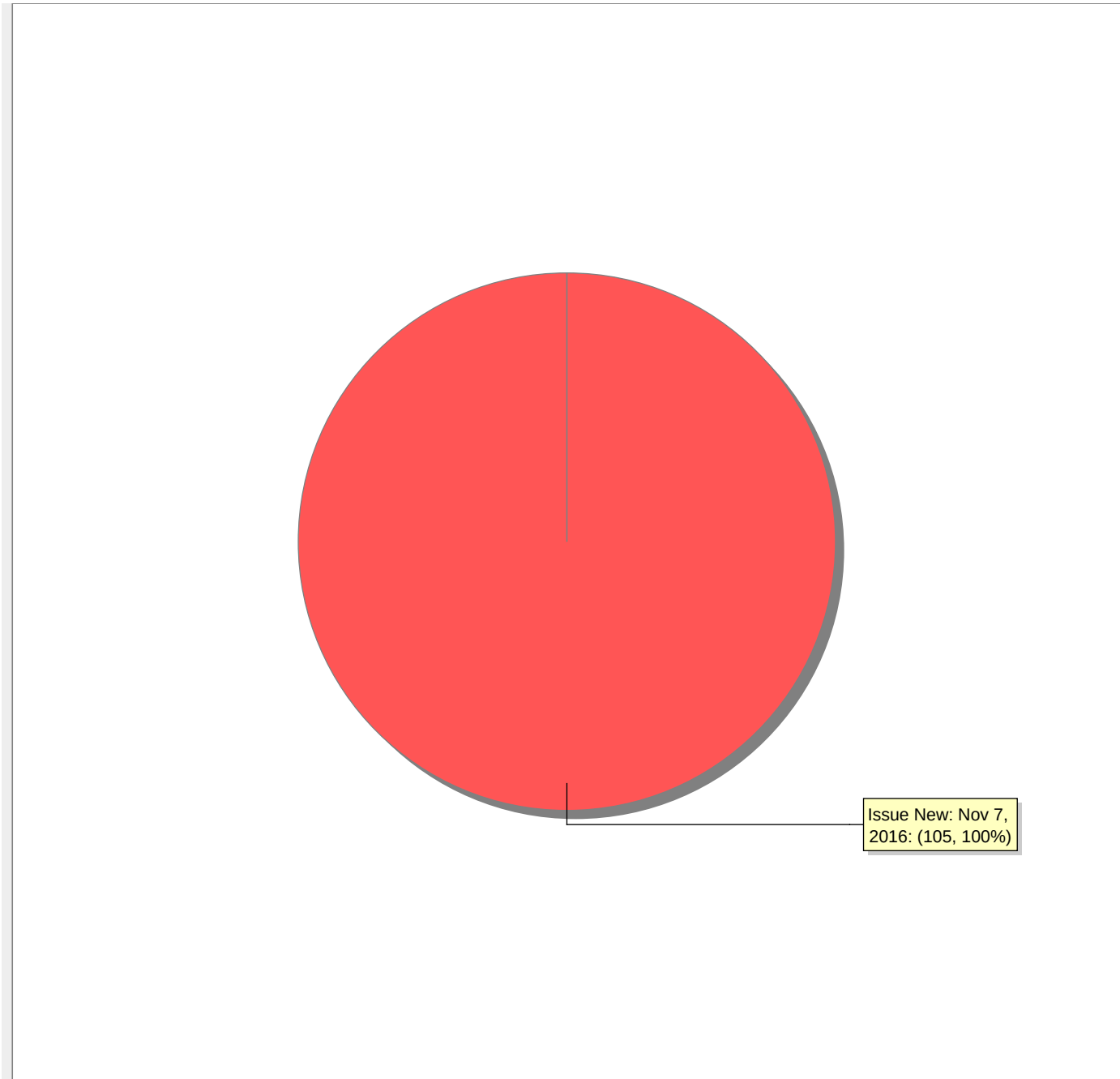


● <none>

New Issues

Issues by New Issue

The following issues have been discovered since the last scan.



Issue New: Nov 7, 2016: (105, 100%)

● Issue New: Nov 7, 2016